Snowflake CSV Ingestion Lab

Complete End-to-End Documentation

Database setup • CSV staging • COPY INTO • Data quality • Analytics • Cleanup

Worksheet: WH_LAB.sql
Database: SASI_RA_DB | Schema: RAW_INGESTION
Warehouse: SASI_RETAIL_INGESTION_WH

Snowflake CSV Ingestion Lab — Complete Documentation

End-to-end practical guide with Snowsight screenshots

Worksheet: `WH_LAB.sql`

Database: `SASI_RA_DB`

Schema: `RAW_INGESTION`

Warehouse: `SASI_RETAIL_INGESTION_WH`

Source file: `orders.csv`

Target table: `SASI_RA_DB.RAW_INGESTION.ORDERS_RAW`

Document scope

This consolidated document brings together all four documentation parts and their supplied Snowsight screenshots. It follows the workflow from session inspection and Snowflake object creation through CSV staging, ingestion, data-quality validation, analytical queries, COPY history inspection, and final resource cleanup.

Workflow at a glance

1. Inspect the active Snowflake session and role.
2. Create and verify the database, schema, and virtual warehouse.
3. Create the raw orders table and inspect its metadata.
4. Define the CSV file format and create an internal stage.
5. Upload and preview `orders.csv`.
6. Load the staged file with `COPY INTO`.
7. Validate row counts, NULL values, and duplicate order IDs.
8. Summarize orders by status and product category.
9. Inspect COPY history.
10. Remove the staged file and drop the lab database and warehouse.

Naming convention

Schema-level objects are referenced using the fully qualified form `DATABASE.SCHEMA.OBJECT`, for example `SASI_RA_DB.RAW_INGESTION.ORDERS_RAW`. Warehouses are account-level compute objects and are referenced by warehouse name.

Consolidated results

Validation	Observed result
Records previewed in stage	20
Rows parsed by `COPY INTO`	20

Validation	Observed result
Rows loaded by `COPY INTO`	20
Errors reported by load	0
Target table row count	20
Duplicate `ORDER_ID` groups	0
NULL values in visible checks	0
COPY history status	Loaded
Staged source file	Removed
Lab database	Successfully dropped
Lab warehouse	Successfully dropped

Snowflake CSV Ingestion Lab — Part 1

Environment inspection and foundational object creation

1. Objective

This part documents the initial Snowflake setup shown in the supplied Snowsight screenshots. The worksheet is named `WH_LAB.sql`.

The work creates a dedicated database, a raw-ingestion schema, and a dedicated virtual warehouse:

- Database: `SASI_RA_DB`
- Schema: `SASI_RA_DB.RAW_INGESTION`
- Warehouse: `SASI_RETAIL_INGESTION_WH`

The screenshots show the active role as `ACCOUNTADMIN`. The initial warehouse is `COMPUTE_WH`; no database or schema is initially selected.

Naming convention: Schema-level objects should be referenced using `DATABASE.SCHEMA.OBJECT`. A warehouse is an account-level compute object and is referenced by its warehouse name, not by a three-part name.

2. Inspect the current Snowflake session

The first query checks the current user, role, warehouse, database, and schema.

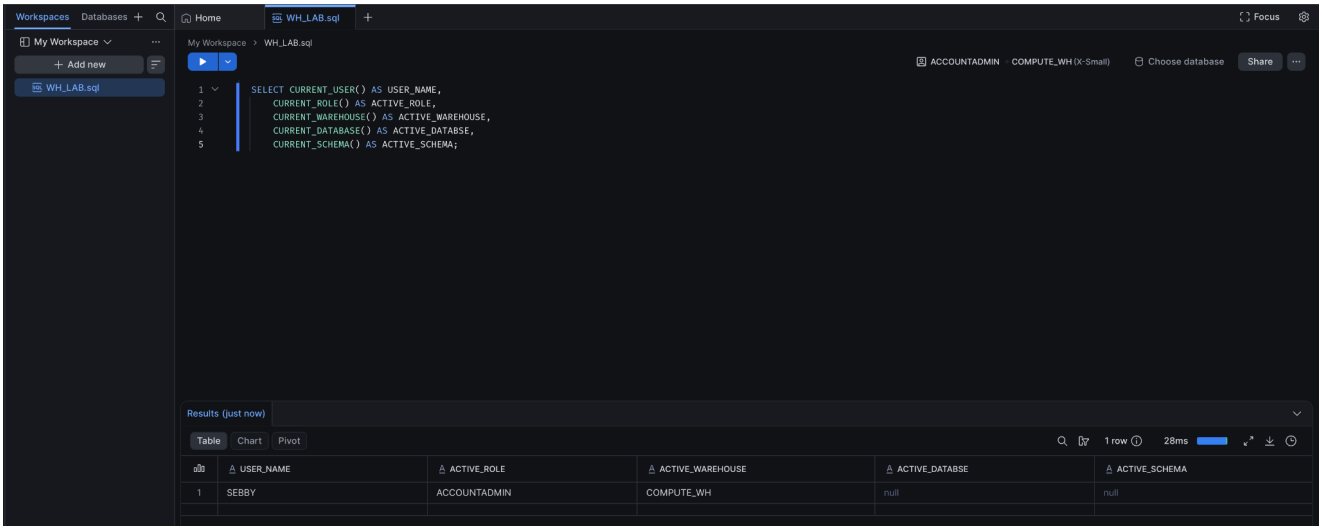
```
SELECT
    CURRENT_USER() AS USER_NAME,
    CURRENT_ROLE() AS ACTIVE_ROLE,
    CURRENT_WAREHOUSE() AS ACTIVE_WAREHOUSE,
    CURRENT_DATABASE() AS ACTIVE_DATABASE,
    CURRENT_SCHEMA() AS ACTIVE_SCHEMA;
```

Observed result

Field	Observed value
User	`SEBBY`
Active role	`ACCOUNTADMIN`

Field	Observed value
Active warehouse	`COMPUTE_WH`
Active database	`NULL`
Active schema	`NULL`

This establishes the starting session context.



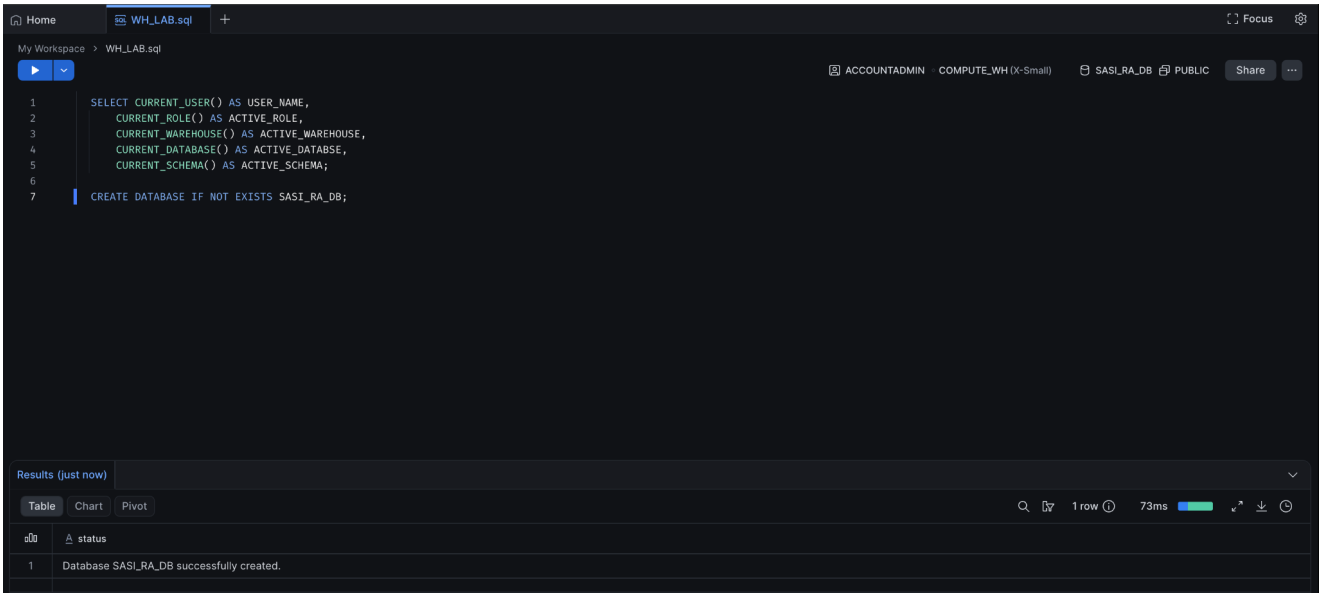
Initial session context

3. Create the database

```
CREATE DATABASE IF NOT EXISTS SASI_RA_DB;
```

`IF NOT EXISTS` allows the statement to be rerun without failing if the database already exists. It does not reset an existing database.

Observed result: `Database SASI_RA_DB successfully created.`



Create database

4. Create the raw-ingestion schema

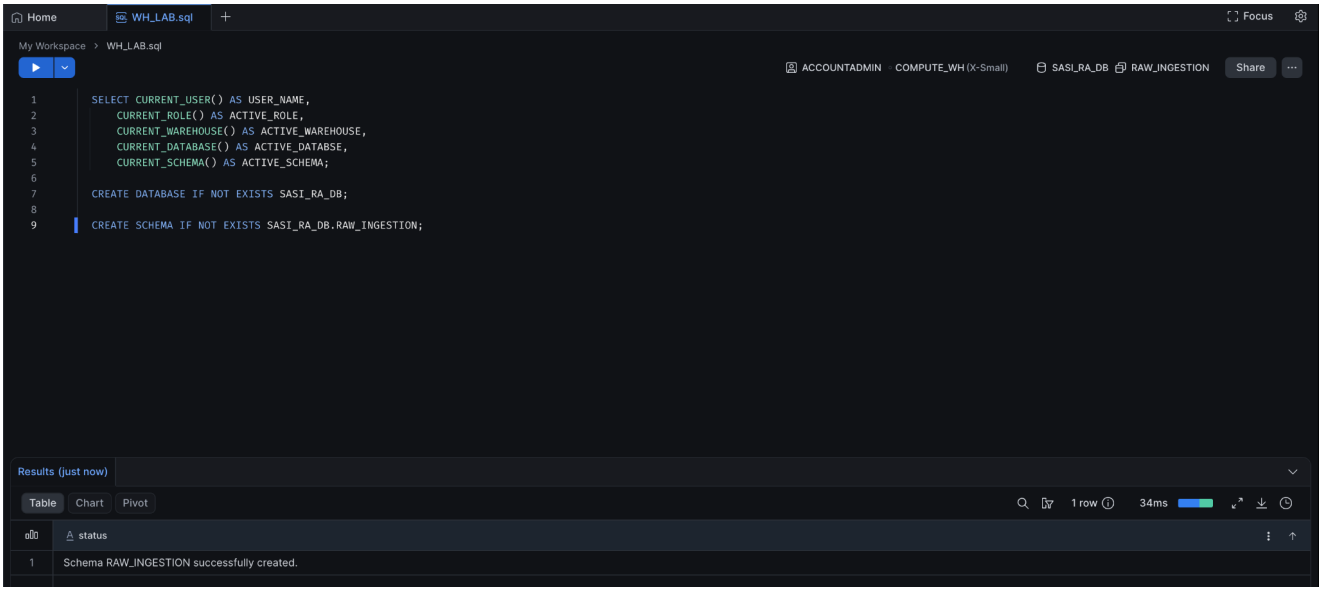
Create the schema inside the database using its fully qualified name:

```
CREATE SCHEMA IF NOT EXISTS SASI_RA_DB.RAW_INGESTION;
```

The naming pattern for schema-level objects in this lab is:

```
SASI_RA_DB.RAW_INGESTION.<OBJECT_NAME>
```

Observed result: `Schema RAW_INGESTION successfully created.`



Create schema

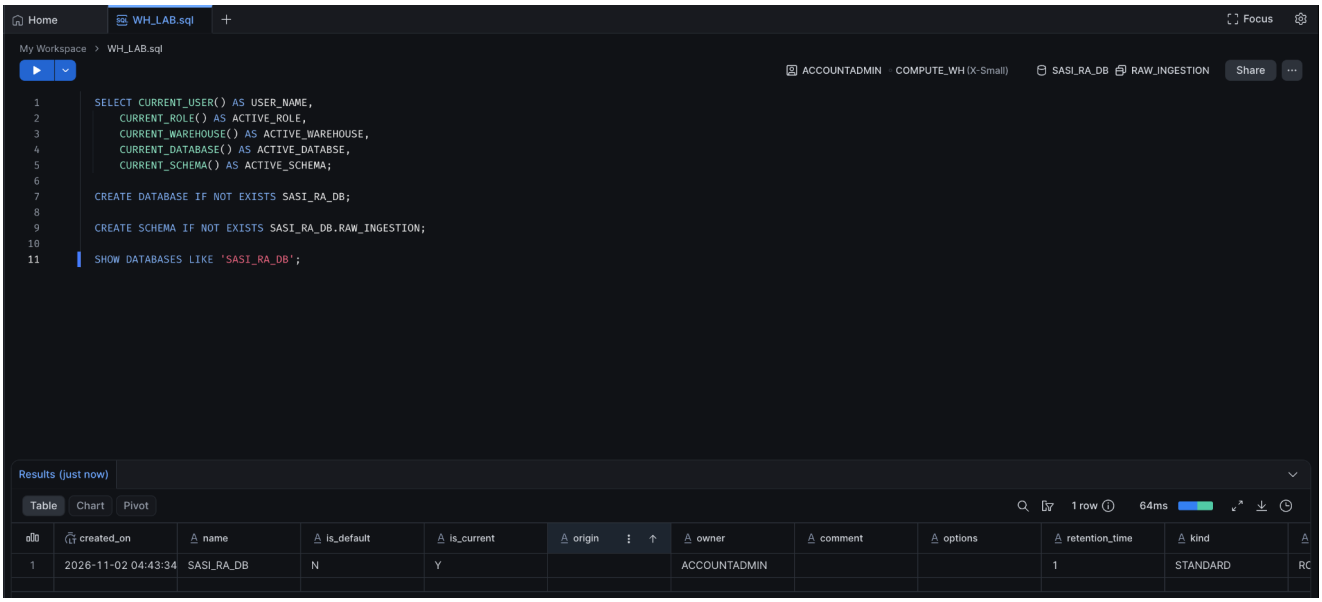
5. Verify the database and schema

```
SHOW DATABASES LIKE 'SASI_RA_DB';

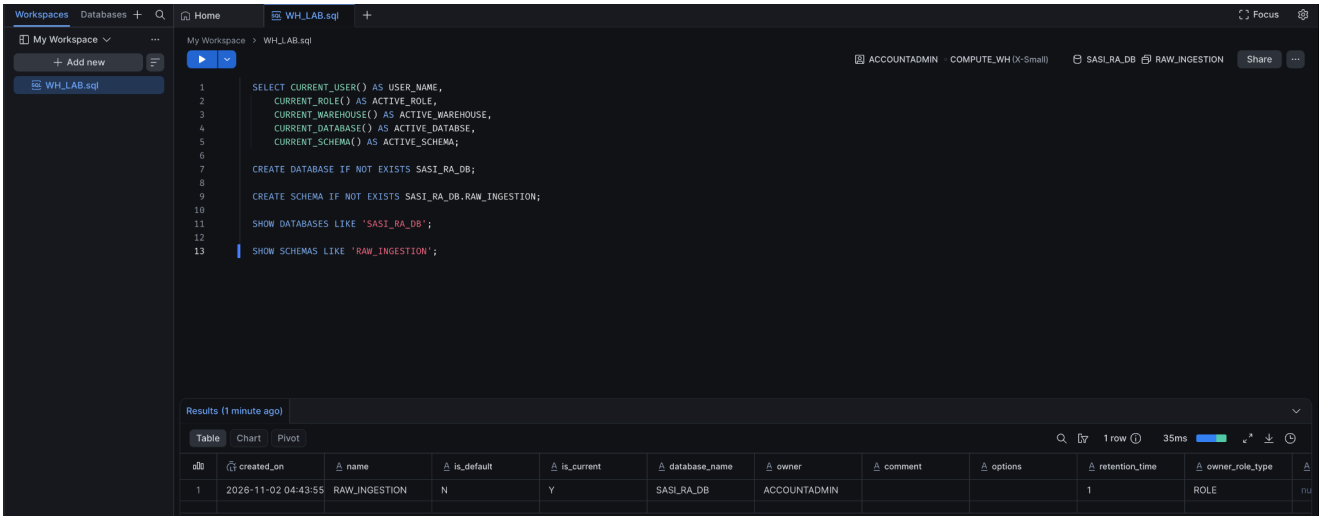
SHOW SCHEMAS LIKE 'RAW_INGESTION';
```

The database listing returned `SASI_RA_DB`. The schema listing returned `RAW_INGESTION`, with `SASI_RA_DB` shown as its parent database.

`SHOW` commands discover objects within a supported scope. To list tables in a particular schema, for example, use `SHOW TABLES IN SCHEMA database_name.schema_name`.



Show database



Show schema

6. Create and select the virtual warehouse

```
CREATE WAREHOUSE IF NOT EXISTS SASI_RETAIL_INGESTION_WH
    WAREHOUSE_SIZE = 'XSMALL'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE;
```

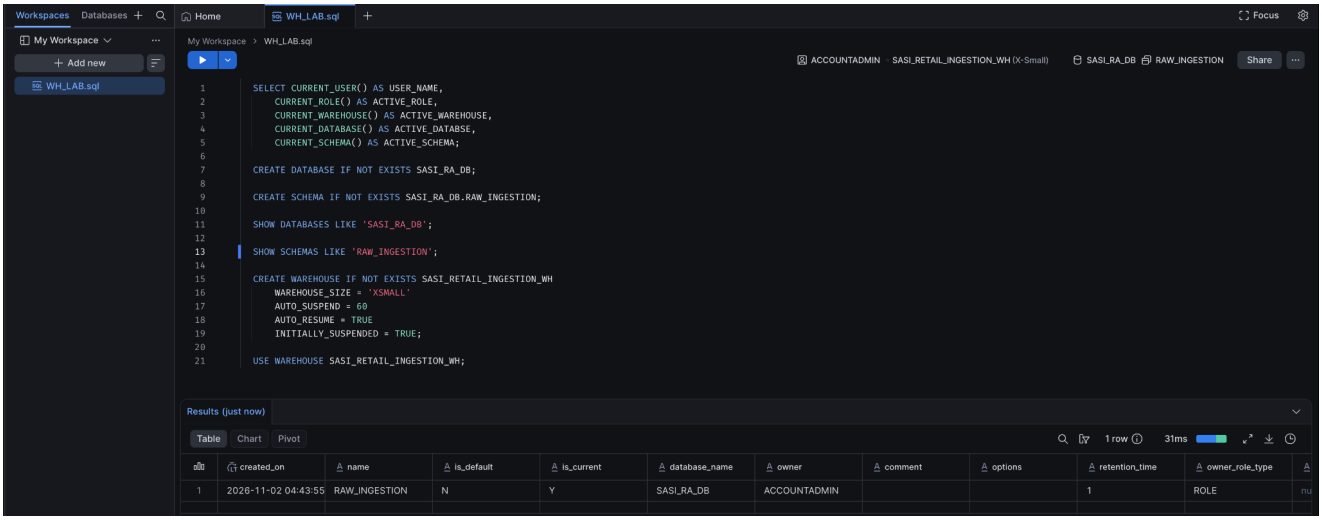
Configuration

Property	Value	Purpose
`WAREHOUSE_SIZE`	`XSMALL`	Small compute size for this practice workload
`AUTO_SUSPEND`	`60`	Suspends after 60 seconds of inactivity
`AUTO_RESUME`	`TRUE`	Allows automatic resumption when needed
`INITIALLY_SUSPENDED`	`TRUE`	Creates the warehouse in a suspended state

Select it for the current worksheet session:

```
USE WAREHOUSE SASI_RETAIL_INGESTION_WH;
```

A warehouse is an account-level object, so it is referenced by name rather than as `DATABASE.SCHEMA.WAREHOUSE`.



Create and select warehouse

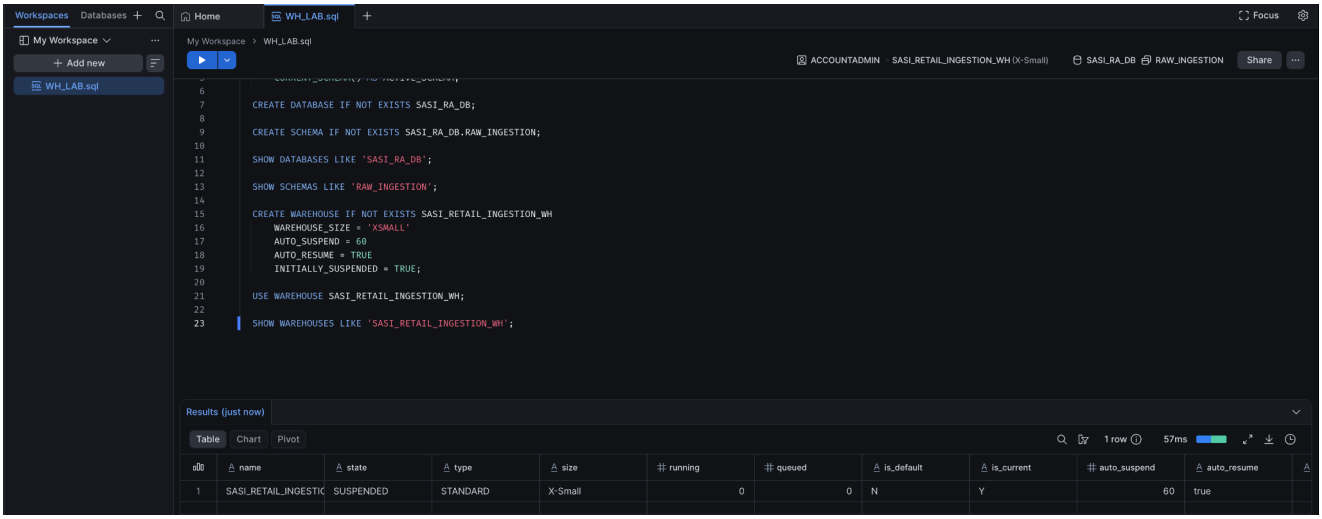
7. Verify the warehouse

```
SHOW WAREHOUSES LIKE 'SASI_RETAIL_INGESTION_WH';
```

The screenshot shows the following configuration:

- Name: `SASI_RETAIL_INGESTION_WH`
- State: `SUSPENDED`
- Type: `STANDARD`
- Size: `X-Small`
- Auto-suspend: `60`
- Auto-resume: `TRUE`

The suspended state is expected because `INITIALLY_SUSPENDED = TRUE` was specified.

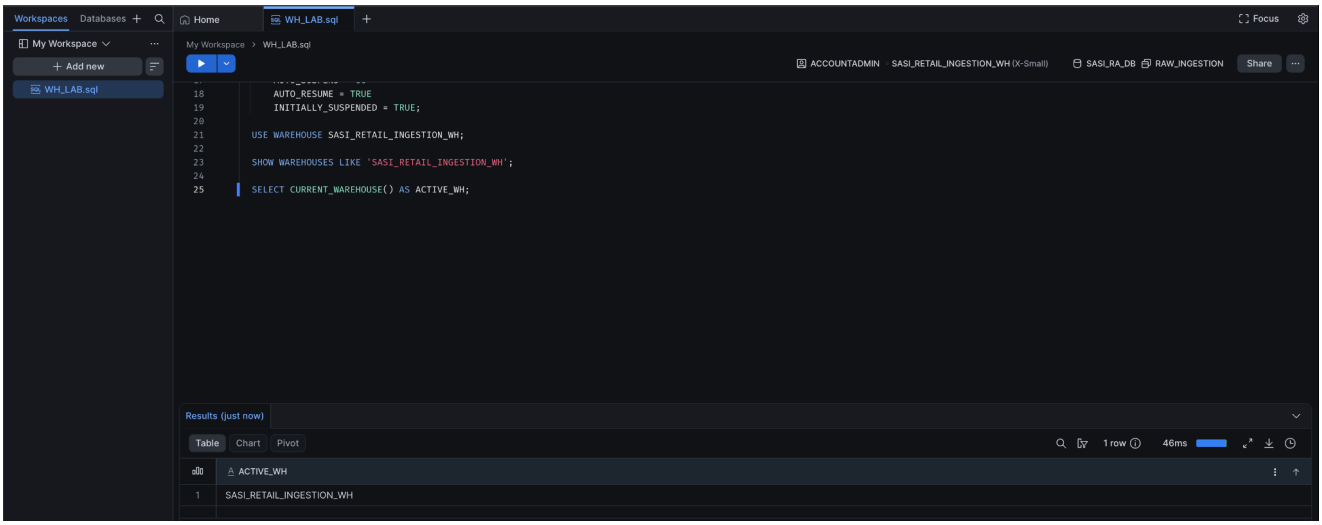


Show warehouse

Confirm the active warehouse in the worksheet session:

```
SELECT CURRENT_WAREHOUSE() AS ACTIVE_WH;
```

Observed result: `SASI_RETAIL_INGESTION_WH`



Verify active warehouse

8. Part 1 outcome

Object type	Object name	Status evidenced by screenshots
Database	`SASI_RA_DB`	Created and listed
Schema	`SASI_RA_DB.RAW_INGESTION`	Created and listed
Warehouse	`SASI_RETAIL_INGESTION_WH`	Created, listed, and selected

Conventions established

- Use descriptive uppercase snake-case object names.
- Fully qualify schema-level objects as `DATABASE.SCHEMA.OBJECT`.
- Use explicit scope with `SHOW` commands.
- Reference warehouses by their account-level name.
- Use `IF NOT EXISTS` for repeatable object creation where appropriate.

Next part

Part 2 will document the next operations shown in the next screenshot batch, such as table creation, file-format configuration, stage creation, file upload, and loading—depending on the screenshots provided.

This section records the SQL and visible outcomes in the supplied screenshots. It does not assume that later ingestion steps have already been completed.

Snowflake CSV Ingestion Lab — Part 2

Raw table, CSV file format, and internal stage

This part continues from Part 1. The database `SASI_RA_DB`, schema `RAW_INGESTION`, and warehouse `SASI_RETAIL_INGESTION_WH` have already been created.

All schema-level objects in this part use the fully qualified naming convention:

```
SASI_RA_DB.RAW_INGESTION.OBJECT_NAME
```

9. Create the raw orders table

The target table is `ORDERS_RAW`. It is created in the `RAW_INGESTION` schema:

```
CREATE TABLE IF NOT EXISTS
    SASI_RA_DB.RAW_INGESTION.ORDERS_RAW
(
    ORDER_ID NUMBER(10, 0),
    CUSTOMER_ID NUMBER(10, 0),
    ORDER_DATE DATE,
    PRODUCT_CATEGORY VARCHAR(100),
    ORDER_AMOUNT NUMBER(12, 2),
    PAYMENT_METHOD VARCHAR(50),
    ORDER_STATUS VARCHAR(30),
    CUSTOMER_CITY VARCHAR(100)
);
```

Column definitions

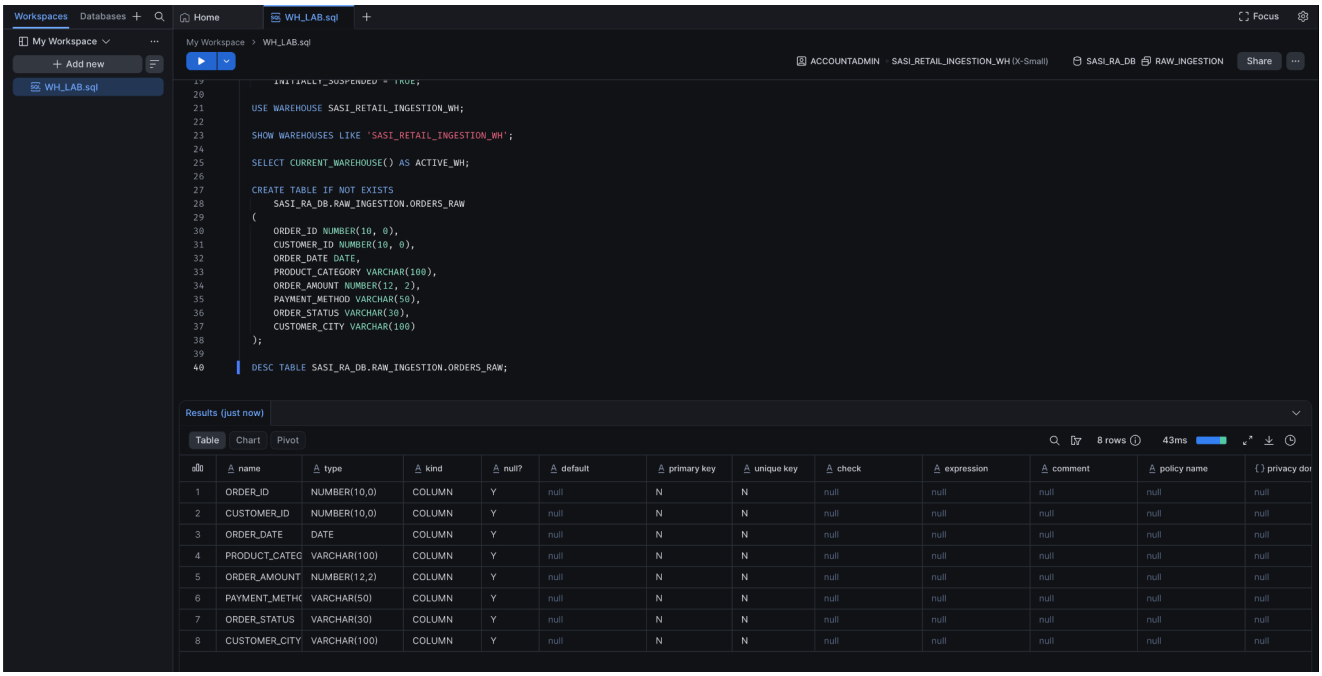
Column	Data type	Purpose
`ORDER_ID`	`NUMBER(10,0)`	Order identifier
`CUSTOMER_ID`	`NUMBER(10,0)`	Customer identifier
`ORDER_DATE`	`DATE`	Order date
`PRODUCT_CATEGORY`	`VARCHAR(100)`	Product category
`ORDER_AMOUNT`	`NUMBER(12,2)`	Monetary order amount with two decimal places
`PAYMENT_METHOD`	`VARCHAR(50)`	Payment method
`ORDER_STATUS`	`VARCHAR(30)`	Order status
`CUSTOMER_CITY`	`VARCHAR(100)`	Customer city

`NUMBER(12,2)` is used for the order amount so values are stored with fixed decimal precision rather than approximate floating-point representation.

Describe the table

```
DESC TABLE SASI_RA_DB.RAW_INGESTION.ORDERS_RAW;
```

The result shows eight columns with the declared data types. The screenshot also shows that the columns are nullable and that no primary key or unique key constraint was declared.

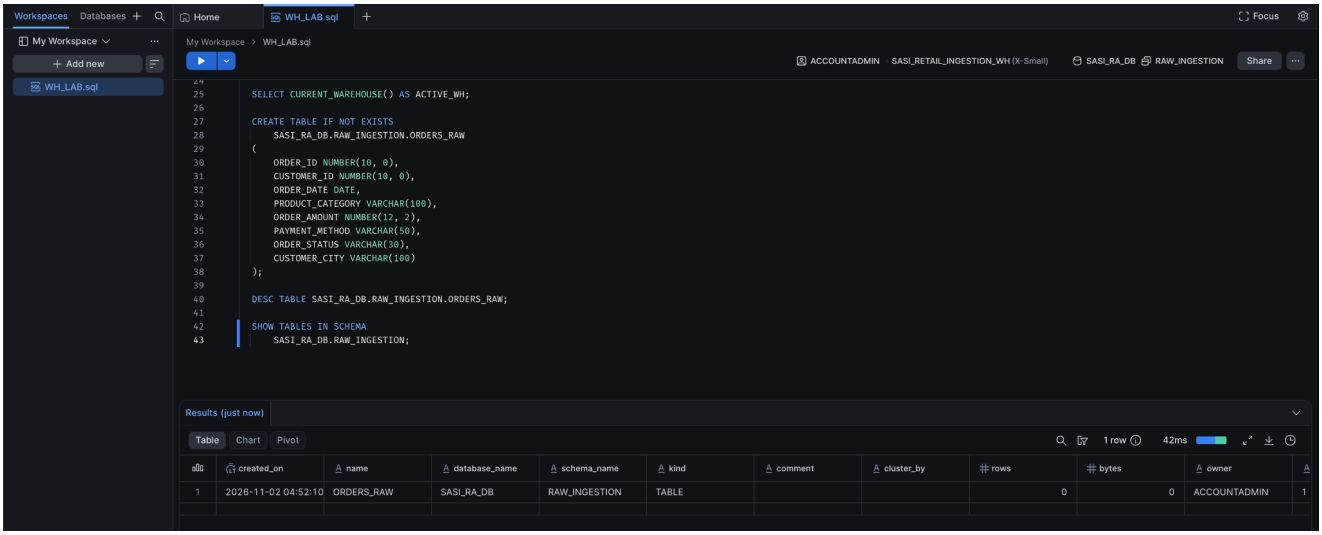


Create and describe raw orders table

10. Verify the table within its schema

```
SHOW TABLES IN SCHEMA
    SASI_RA_DB.RAW_INGESTION;
```

The result lists `ORDERS_RAW` under database `SASI_RA_DB` and schema `RAW_INGESTION`. Its row count is initially zero because no CSV records have been loaded yet.



Show tables in schema

11. Inspect table metadata using INFORMATION_SCHEMA

Snowflake exposes metadata through `INFORMATION_SCHEMA` views. The following query retrieves metadata for the newly created table:

```
SELECT
    TABLE_CATALOG,
```

```
    TABLE_SCHEMA,
    TABLE_NAME,
    ROW_COUNT,
    BYTES,
    CREATED
FROM SASI_RA_DB.INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA = 'RAW_INGESTION'
  AND TABLE_NAME = 'ORDERS_RAW';
```

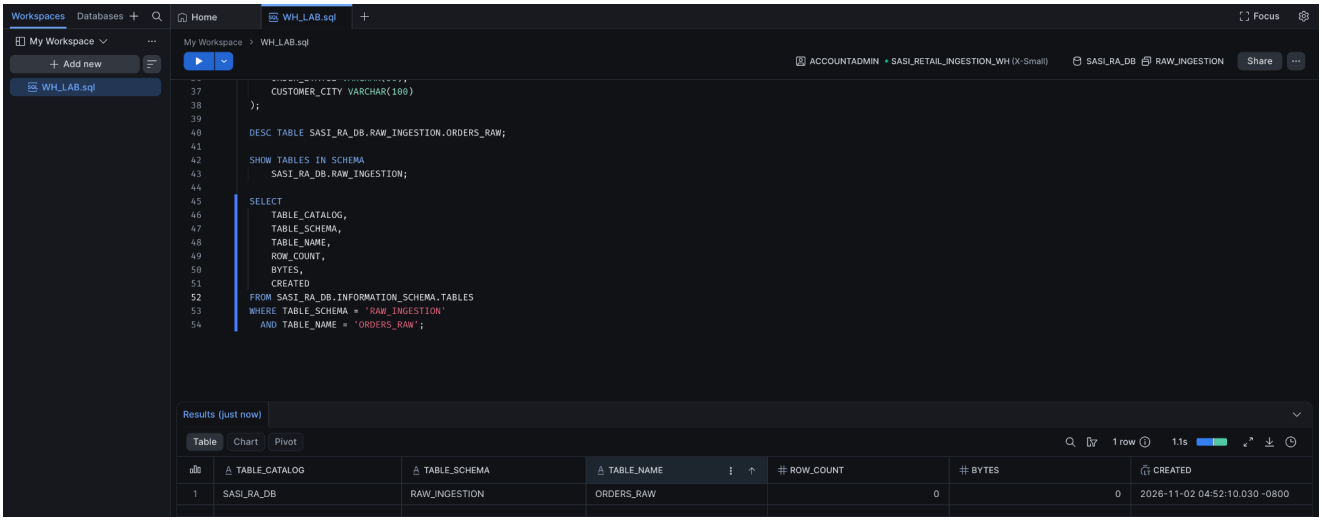
What the columns mean

Metadata column	Meaning
`TABLE_CATALOG`	Database containing the table
`TABLE_SCHEMA`	Schema containing the table
`TABLE_NAME`	Table name
`ROW_COUNT`	Recorded row count
`BYTES`	Storage size in bytes
`CREATED`	Table creation timestamp

The screenshot confirms:

- Catalog: `SASI_RA_DB`
- Schema: `RAW_INGESTION`
- Table: `ORDERS_RAW`
- Row count: `0`
- Bytes: `0`

This is expected before ingestion.



Query information schema tables

12. Create the CSV file format

A file format defines how Snowflake parses the source CSV file.

Create the named file format:

```
CREATE FILE FORMAT IF NOT EXISTS
    SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT
    TYPE = CSV
```

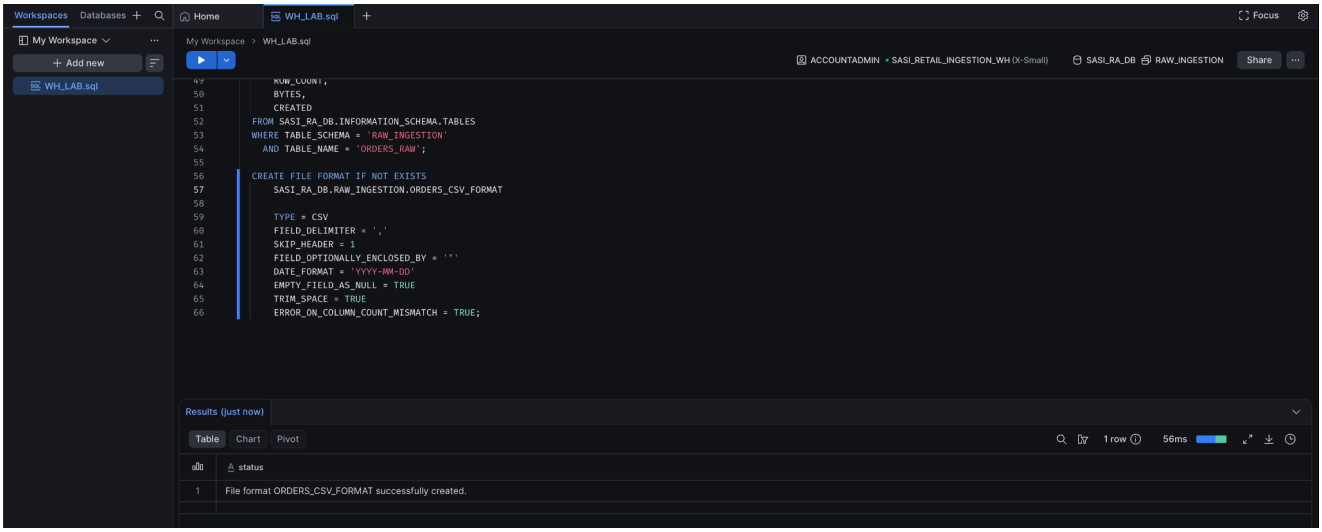
```
    FIELD_DELIMITER = ','
    SKIP_HEADER = 1
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
    DATE_FORMAT = 'YYYY-MM-DD'
    EMPTY_FIELD_AS_NULL = TRUE
    TRIM_SPACE = TRUE
    ERROR_ON_COLUMN_COUNT_MISMATCH = TRUE;
```

Configuration explained

Property	Value	Purpose
`TYPE`	`CSV`	Specifies comma-separated input
`FIELD_DELIMITER`	`,`	Separates fields using commas
`SKIP_HEADER`	`1`	Skips the first row containing column headings
`FIELD_OPTIONALLY_ENCLOSED_BY`	`"`	Handles optionally quoted CSV fields
`DATE_FORMAT`	`YYYY-MM-DD`	Defines the expected date representation
`EMPTY_FIELD_AS_NULL`	`TRUE`	Treats empty fields as NULL
`TRIM_SPACE`	`TRUE`	Trims surrounding spaces
`ERROR_ON_COLUMN_COUNT_MISMATCH`	`TRUE`	Treats inconsistent field counts as an error

The command returned:

```
File format ORDERS_CSV_FORMAT successfully created.
```



Create CSV file format

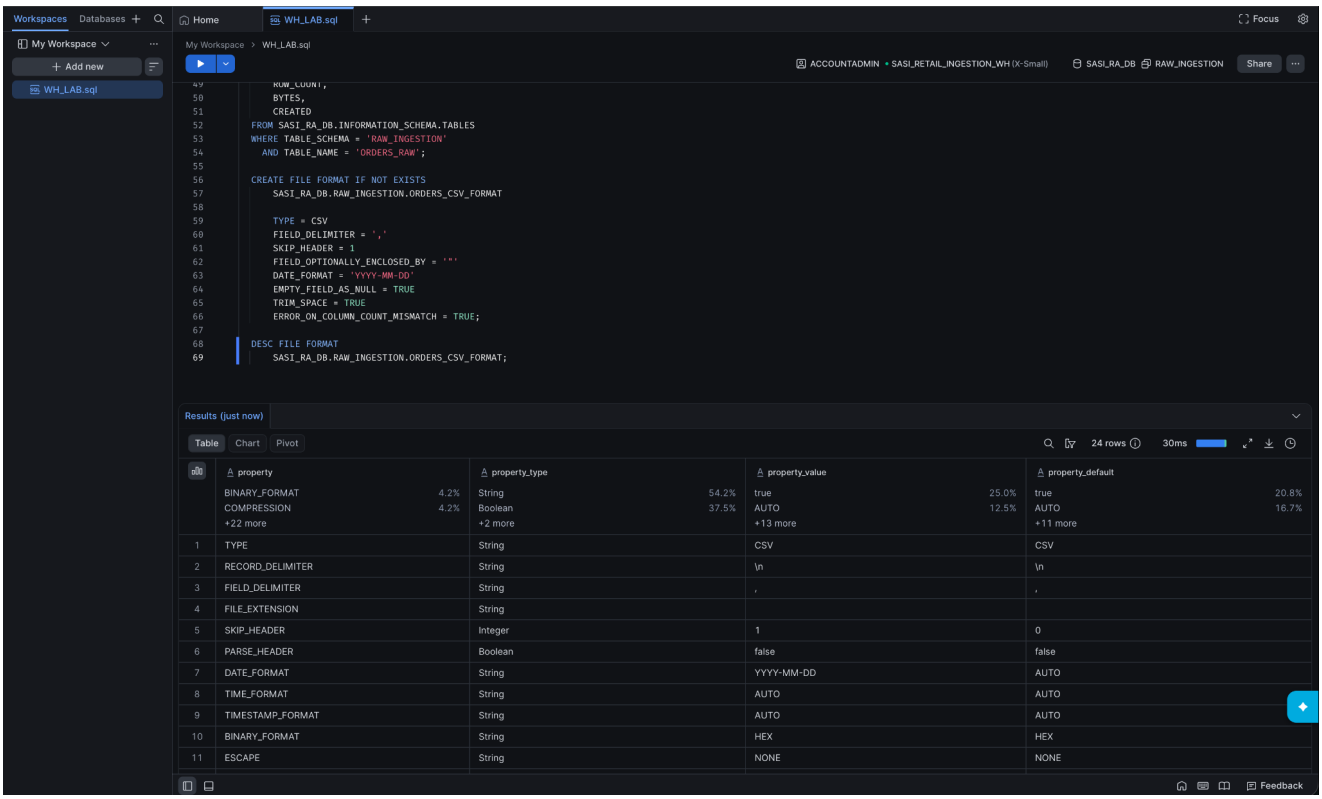
13. Describe the CSV file format

```
DESC FILE FORMAT
    SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT;
```

The result displays the file format's properties, including:

- CSV type
- Comma field delimiter
- Header skipping set to `1`
- Date format set to `YYYY-MM-DD`
- Empty-field handling enabled

- Other format properties and their defaults

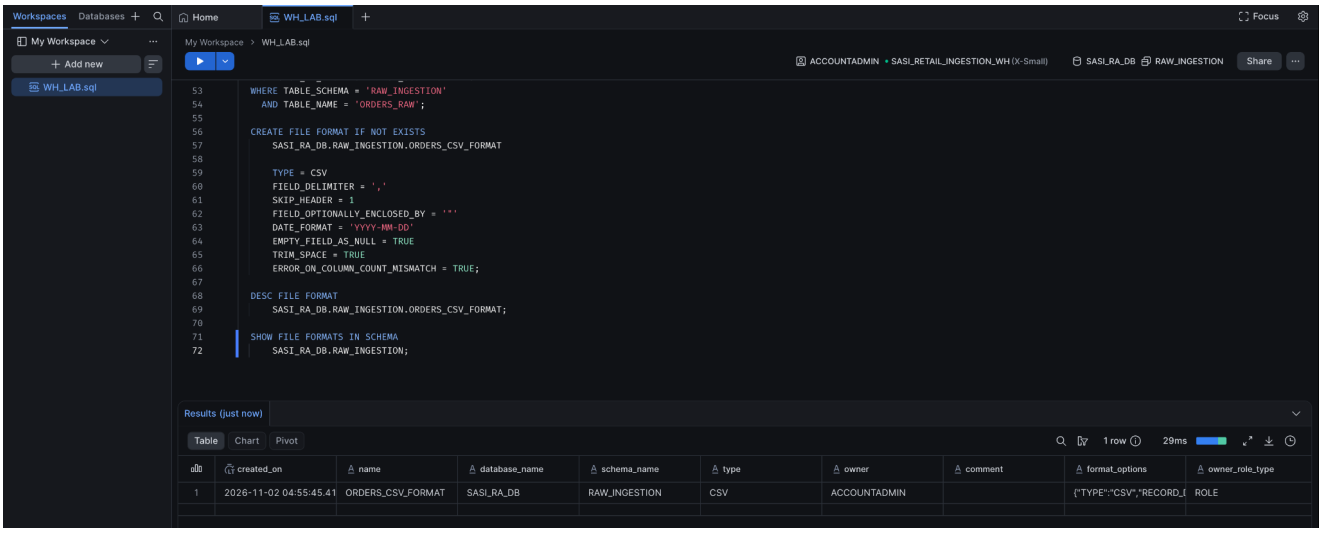


Describe CSV file format

14. Verify the file format in the schema

```
SHOW FILE FORMATS IN SCHEMA
    SASI_RA_DB.RAW_INGESTION;
```

The result lists `ORDERS_CSV_FORMAT` and identifies its database, schema, and type.



Show file formats in schema

15. Create the internal stage

A stage provides a Snowflake-managed location for files that will be loaded into tables.

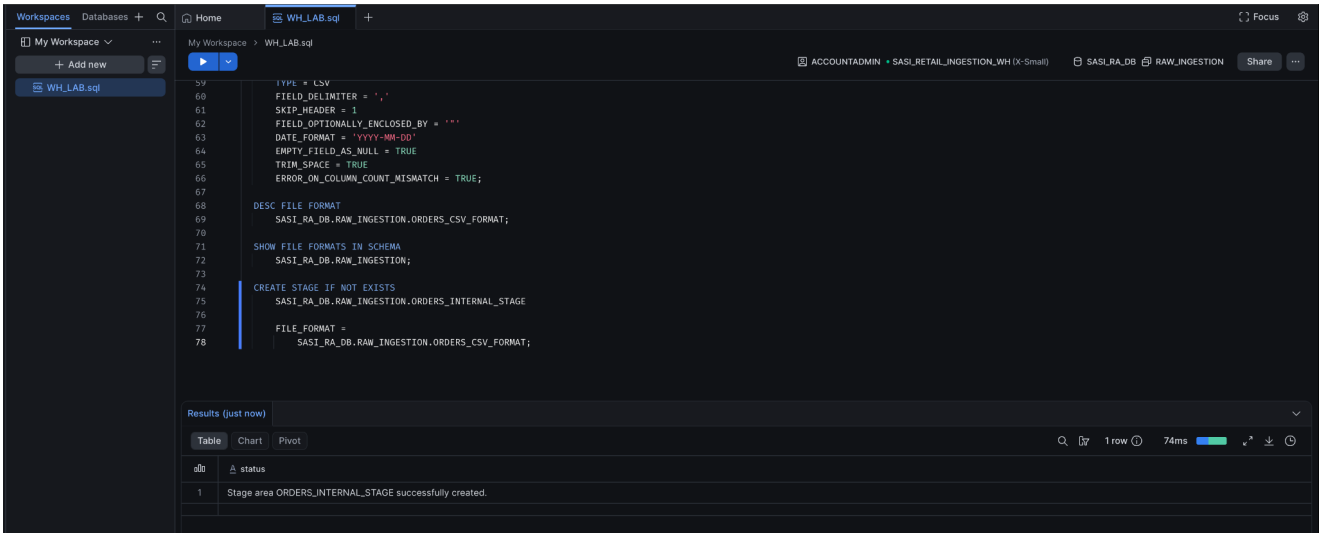
Create the internal stage and associate it with the named CSV file format:

```
CREATE STAGE IF NOT EXISTS
    SASI_RA_DB.RAW_INGESTION.ORDERS_INTERNAL_STAGE
    FILE_FORMAT =
        SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT;
```

The command returned:

```
Stage area ORDERS_INTERNAL_STAGE successfully created.
```

The stage is associated with `ORDERS_CSV_FORMAT`, so the file-format settings can be applied when reading staged files.



Create internal stage

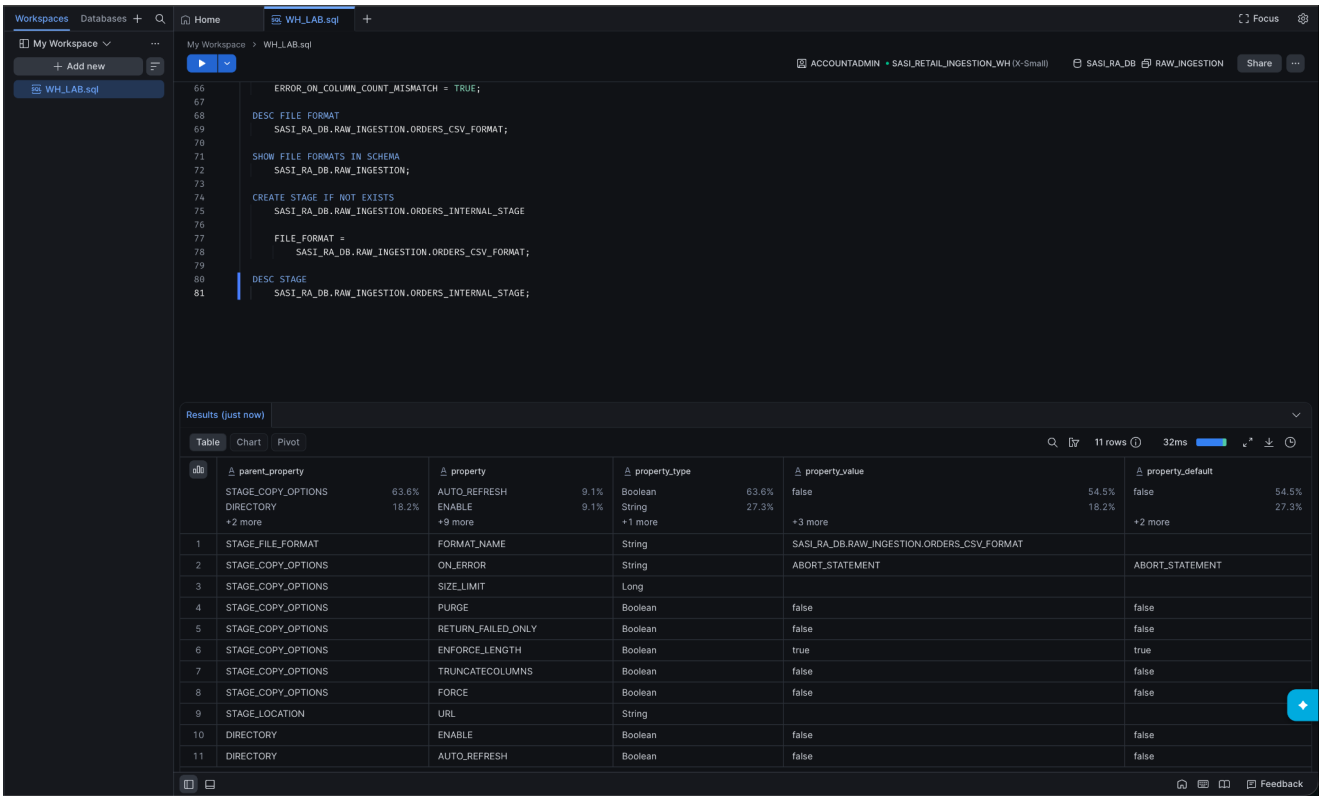
16. Describe the internal stage

```
DESC STAGE
    SASI_RA_DB.RAW_INGESTION.ORDERS_INTERNAL_STAGE;
```

The screenshot shows the stage properties, including:

Property	Value shown
STAGE_FILE_FORMAT	SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT
ON_ERROR	ABORT_STATEMENT
PURGE	FALSE
RETURN_FAILED_ONLY	FALSE
ENFORCE_LENGTH	TRUE
TRUNCATECOLUMNS	FALSE
FORCE	FALSE

This verifies that the stage references the intended file format and exposes its copy-related options.



Describe internal stage

17. Part 2 outcome

The following schema-level objects have now been created and inspected:

Object type	Fully qualified name
Table	`SASI_RA_DB.RAW_INGESTION.ORDERS_RAW`
File format	`SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT`
Internal stage	`SASI_RA_DB.RAW_INGESTION.ORDERS_INTERNAL_STAGE`

Ingestion architecture prepared

```
CSV source file
      |
      v
ORDERS_INTERNAL_STAGE
      |
      | uses ORDERS_CSV_FORMAT
      v
ORDERS_RAW
```

At the end of this part, the target table exists but has zero rows. The file format and internal stage have been created and verified. The screenshots in this batch do not yet show a file upload or a data load.

Next part

The next section will document the stage file upload, file listing, staged-file preview, `COPY INTO` execution, and post-load validation as evidenced by the next screenshot batch.

This documentation reflects the SQL statements and visible Snowsight results in the supplied screenshots.

Snowflake CSV Ingestion Lab — Part 3

Stage verification, CSV preview, COPY INTO, and validation

This part continues the lab in database `SASI_RA_DB`, schema `RAW_INGESTION`, using the internal stage `ORDERS_INTERNAL_STAGE`, file format `ORDERS_CSV_FORMAT`, and target table `ORDERS_RAW`.

All schema-level object references remain fully qualified:

```
SASI_RA_DB.RAW_INGESTION.OBJECT_NAME
```

17. Verify the internal stage

First, list the stages in the target schema:

```
SHOW STAGES IN SCHEMA
    SASI_RA_DB.RAW_INGESTION;
```

The result lists `ORDERS_INTERNAL_STAGE` in `SASI_RA_DB.RAW_INGESTION`.

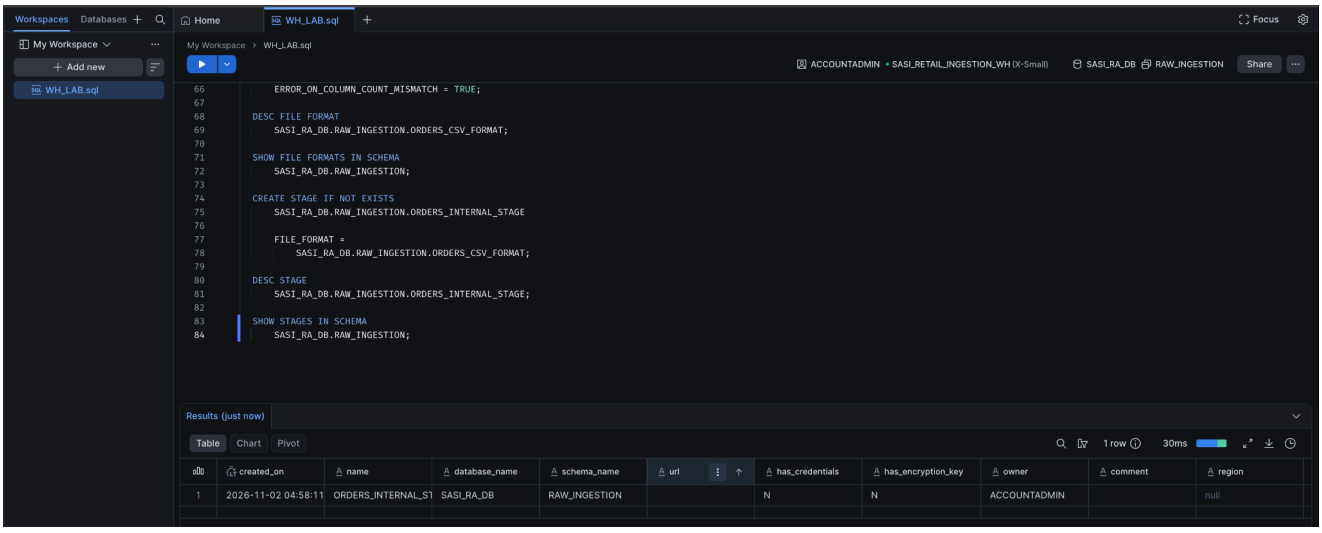
Then list the files stored in the stage:

```
LIST @SASI_RA_DB.RAW_INGESTION.ORDERS_INTERNAL_STAGE;
```

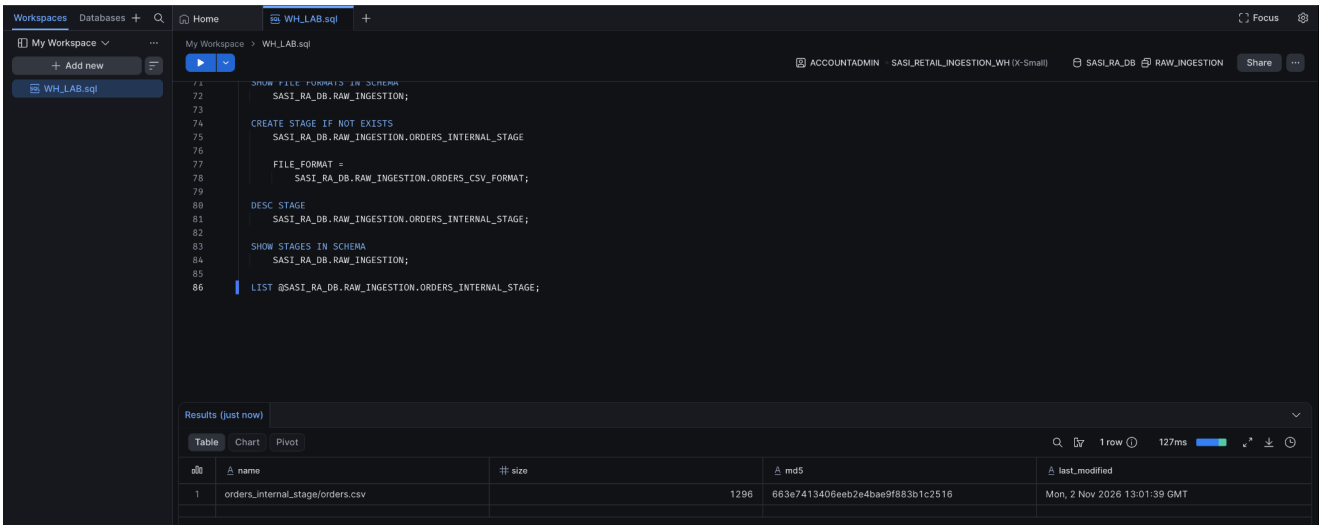
The result shows the uploaded file:

- File: `orders_internal_stage/orders.csv`
- Size: `1,296` bytes

The file's presence confirms that the CSV has been uploaded to the internal stage and is ready to be inspected.



Verify stage and list files



List staged orders CSV

18. Preview the staged CSV data

Before loading the file into the target table, query the staged file directly. The staged file is read using the named CSV file format.

```
SELECT
    $1::NUMBER(10,0) AS ORDER_ID,
    $2::NUMBER(10,0) AS CUSTOMER_ID,
    $3::DATE AS ORDER_DATE,
    $4::VARCHAR(100) AS PRODUCT_CATEGORY,
    $5::NUMBER(12,2) AS ORDER_AMOUNT,
    $6::VARCHAR(50) AS PAYMENT_METHOD,
    $7::VARCHAR(30) AS ORDER_STATUS,
    $8::VARCHAR(100) AS CUSTOMER_CITY
FROM @SASI_RA_DB.RAW_INGESTION.ORDERS_INTERNAL_STAGE
(
    FILE_FORMAT =>
        'SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT'
);
```

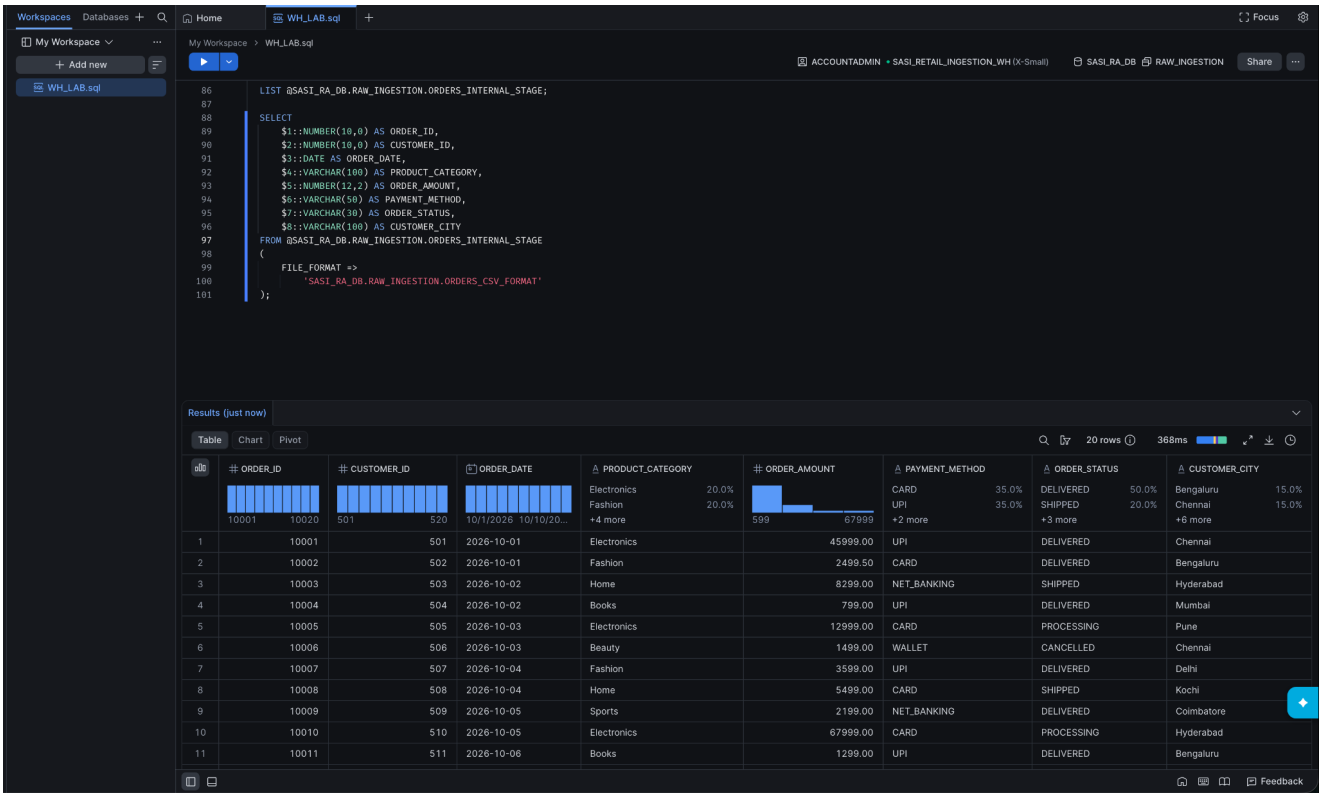
Understanding staged-file column references

When a CSV file is queried directly from a stage, Snowflake exposes its fields positionally:

Reference	Target column
`$1`	`ORDER_ID`
`$2`	`CUSTOMER_ID`
`$3`	`ORDER_DATE`
`$4`	`PRODUCT_CATEGORY`
`$5`	`ORDER_AMOUNT`
`$6`	`PAYMENT_METHOD`
`$7`	`ORDER_STATUS`
`$8`	`CUSTOMER_CITY`

The explicit casts convert the staged text fields into the intended target data types during preview.

The screenshot shows a result containing 20 rows, with sample values such as order IDs `10001` through `10020`, customer IDs `501` through `520`, and dates beginning `2026-10-01`.



Preview staged CSV records

19. Count the staged records

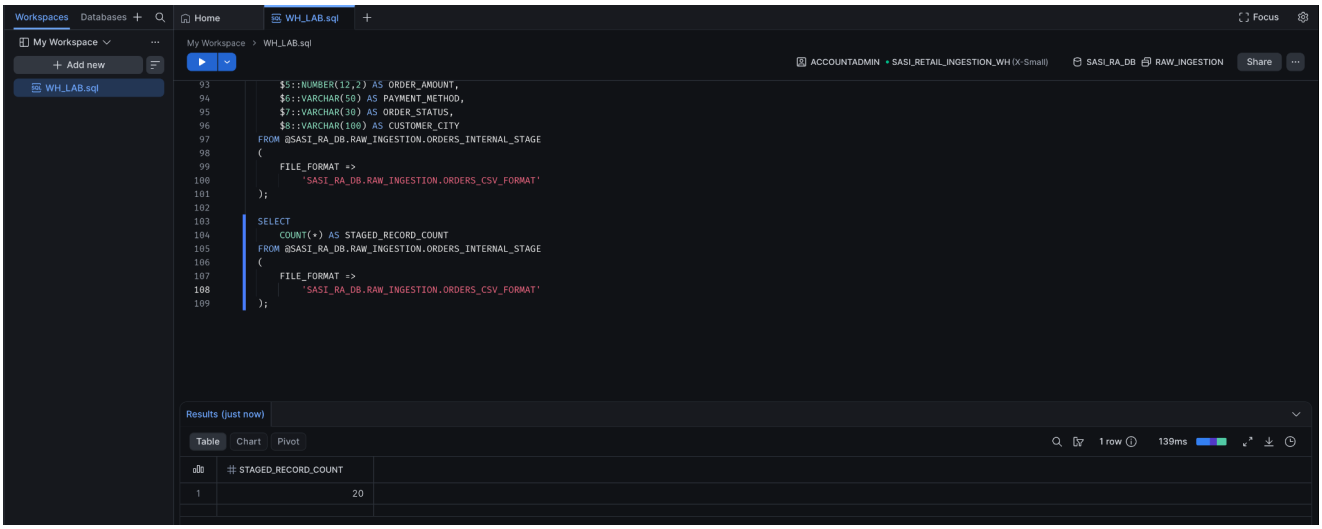
A separate count query checks how many records Snowflake can read from the staged file:

```
SELECT
    COUNT(*) AS STAGED_RECORD_COUNT
FROM @SASI_RA_DB.RAW_INGESTION.ORDERS_INTERNAL_STAGE
(
    FILE_FORMAT =>
        'SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT'
);
```

The result is:

<badge color="success">20 staged records</badge>

This establishes the expected number of data records before loading. Because the file format has `SKIP_HEADER = 1`, the header row is excluded from the data records.



Count staged records

20. Load the CSV into the raw table using COPY INTO

The Snowflake `COPY INTO` command loads data from the internal stage into the target table.

```
COPY INTO
    SASI_RA_DB.RAW_INGESTION.ORDERS_RAW
FROM
    @SASI_RA_DB.RAW_INGESTION.ORDERS_INTERNAL_STAGE
FILE_FORMAT = (
    FORMAT_NAME =
        'SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT'
)
ON_ERROR = 'ABORT_STATEMENT';
```

How this statement works

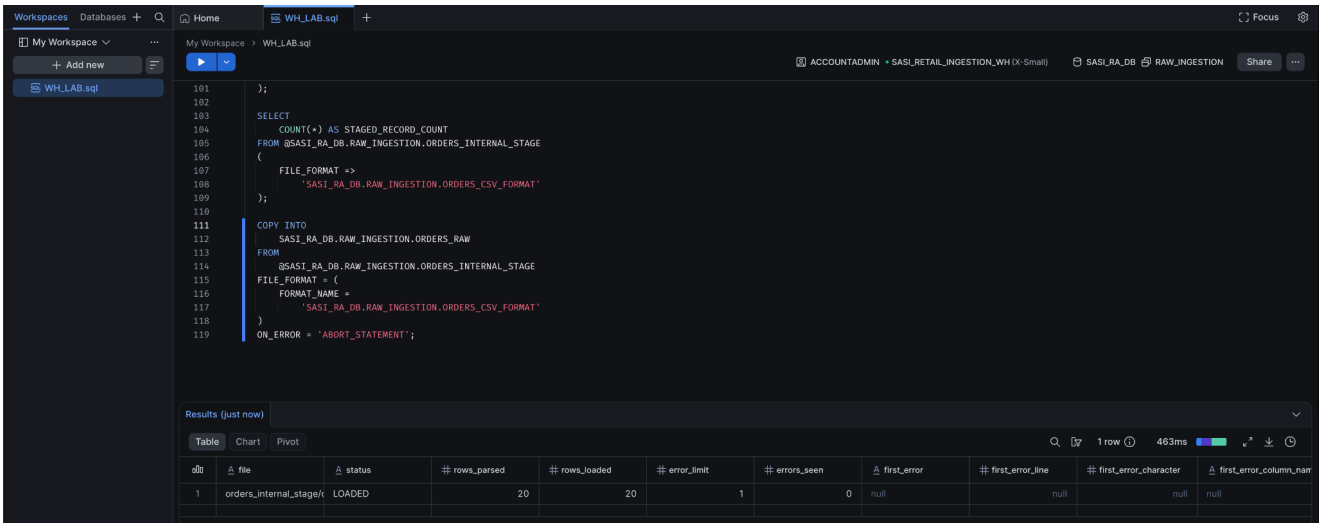
- `COPY INTO`: initiates a bulk load into a Snowflake table.
- Target table: `SASI_RA_DB.RAW_INGESTION.ORDERS_RAW`.
- `FROM`: identifies the internal stage containing the source file.
- `FILE_FORMAT`: applies the named CSV parsing configuration.
- `ON_ERROR = 'ABORT_STATEMENT'`: stops the load statement if a data-loading error occurs, rather than continuing with partial loading.

Load result

The screenshot shows:

Result field	Value
Status	`LOADED`
Rows parsed	20
Rows loaded	20
Errors seen	0

All 20 data rows were loaded successfully.



COPY INTO load result

21. Validate the target table row count

After loading, query the target table:

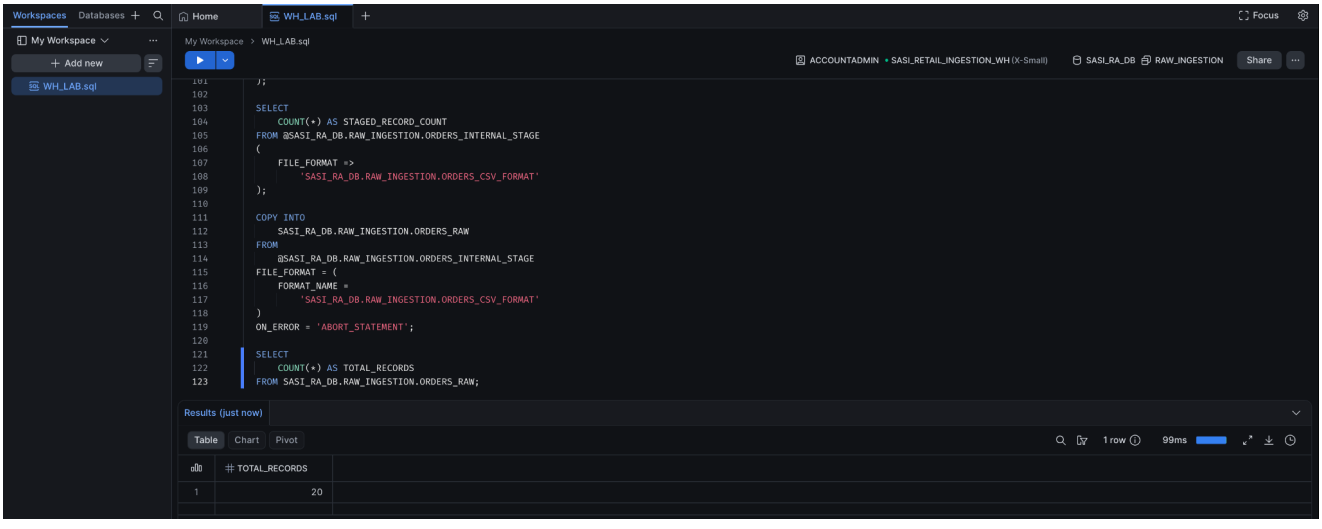
```
SELECT
    COUNT(*) AS TOTAL_RECORDS
FROM SASI_RA_DB.RAW_INGESTION.ORDERS_RAW;
```

The result is:

<badge color="success">20 records</badge>

The target table count matches the staged record count and the `COPY INTO` load result.

This is an important ingestion validation: it checks that the expected number of records is present in the destination table.



Validate loaded record count

22. Inspect all loaded records

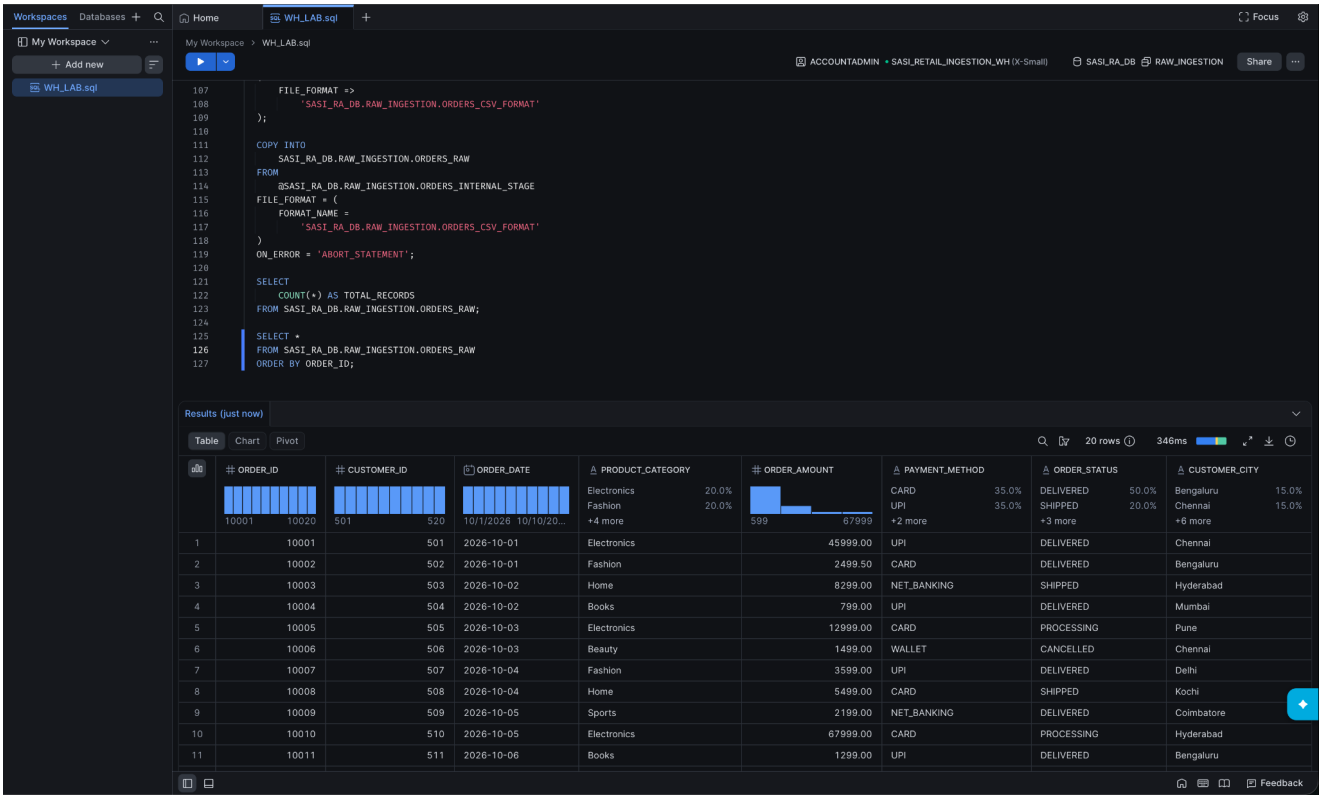
Retrieve the loaded data and sort it by order ID:

```
SELECT *
FROM SASI_RA_DB.RAW_INGESTION.ORDERS_RAW
ORDER BY ORDER_ID;
```

The result displays 20 records across the eight target columns:

- `ORDER_ID`
- `CUSTOMER_ID`
- `ORDER_DATE`
- `PRODUCT_CATEGORY`
- `ORDER_AMOUNT`
- `PAYMENT_METHOD`
- `ORDER_STATUS`
- `CUSTOMER_CITY`

This provides a direct visual check that the data has been inserted into the expected columns.



Query all loaded orders

23. Inspect a smaller sample

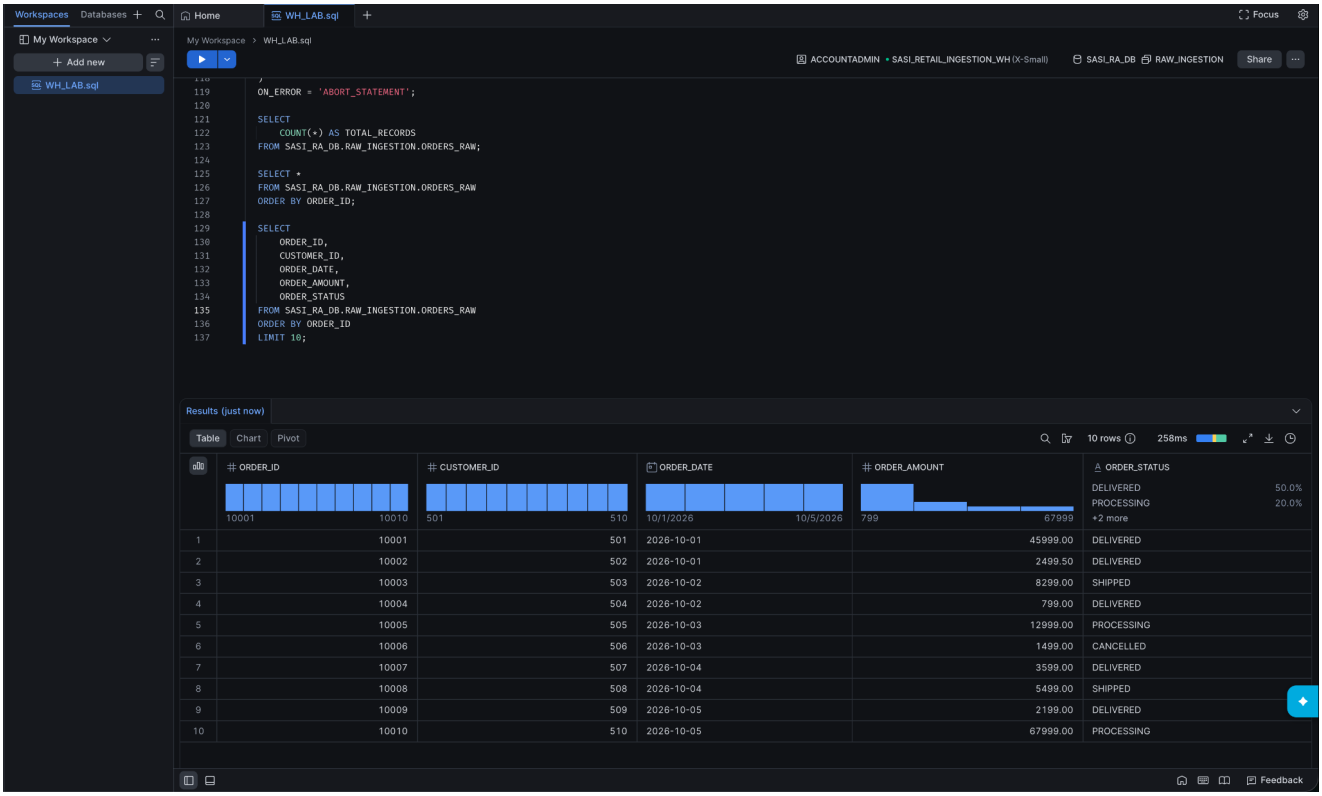
For a more focused inspection, select the key order attributes and limit the result to the first ten records:

```
SELECT
    ORDER_ID,
    CUSTOMER_ID,
    ORDER_DATE,
    ORDER_AMOUNT,
    ORDER_STATUS
FROM SASI_RA_DB.RAW_INGESTION.ORDERS_RAW
ORDER BY ORDER_ID
```

```
LIMIT 10;
```

The result returns ten rows, from order ID `10001` through `10010`.

This is useful for spot-checking identifiers, dates, monetary values, and order statuses without displaying every column or record.



Query first ten orders

24. Part 3 outcome

The CSV ingestion workflow completed successfully.

Validation	Expected	Observed
File present in stage	Yes	Yes
Staged data records	20	20
Rows parsed by COPY INTO	20	20
Rows loaded by COPY INTO	20	20
Errors seen	0	0
Target table row count	20	20

End-to-end ingestion flow

```
orders.csv
    |
    v
ORDERS_INTERNAL_STAGE
    |
    v
ORDERS_CSV_FORMAT
    |
    v
Staged-file preview and record count
```

```
    |
    v
COPY INTO
    |
    v
ORDERS_RAW
    |
    v
COUNT(*) and SELECT validation
```

What this lab demonstrates

1. Creating a Snowflake database, schema, warehouse, and target table.
2. Defining a reusable CSV file format.
3. Creating and inspecting an internal stage.
4. Uploading and listing a source CSV file.
5. Previewing staged data before ingestion.
6. Loading staged data into a table using `COPY INTO`.
7. Validating the load through command output, row counts, and sample queries.
8. Using fully qualified object names to make SQL references explicit.

Important operational note

The screenshots show a successful load of 20 records. They do not by themselves establish that the source data is free of duplicates or business-rule violations. For a production pipeline, add checks such as duplicate order IDs, required-field null checks, valid status/category values, and reconciliation against source totals.

This documentation reflects the SQL statements and visible Snowsight results in the supplied screenshots.

Snowflake CSV Ingestion Lab — Part 4

Data-quality validation, analytical summaries, load history, and cleanup

This section continues the ingestion exercise using the raw table:

```
SASI_RA_DB.RAW_INGESTION.ORDERS_RAW
```

The work in this part checks the loaded data, produces simple business summaries, reviews the file-load history, and removes the staged source file.

25. Check for NULL values

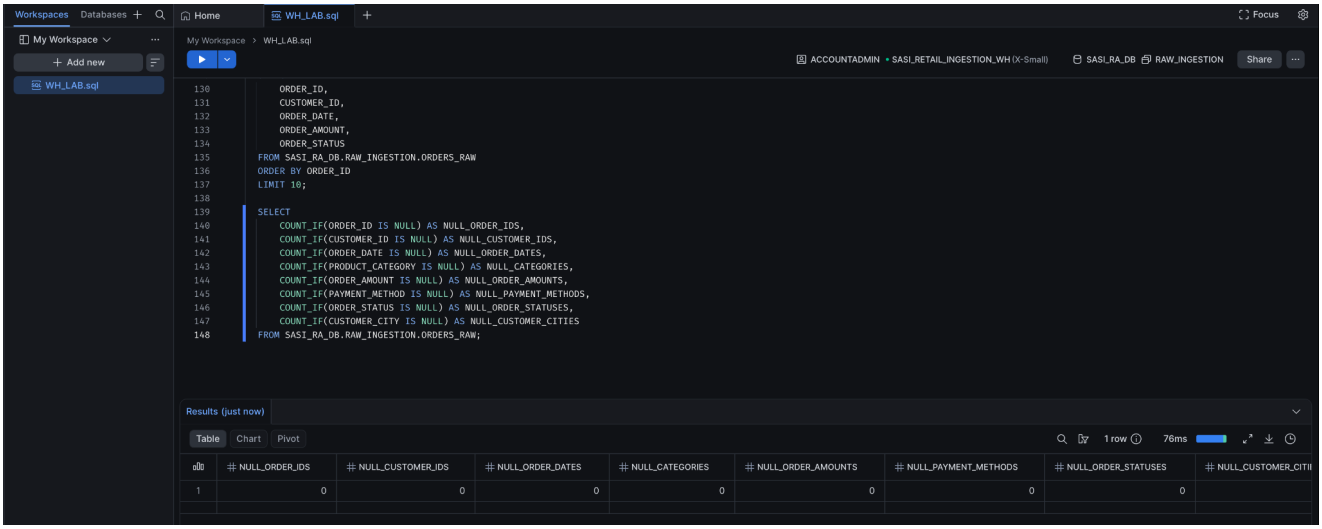
A successful load does not automatically mean every field contains a valid value. The following query counts NULL values independently for each column:

```
SELECT
    COUNT_IF(ORDER_ID IS NULL) AS NULL_ORDER_IDS,
    COUNT_IF(CUSTOMER_ID IS NULL) AS NULL_CUSTOMER_IDS,
    COUNT_IF(ORDER_DATE IS NULL) AS NULL_ORDER_DATES,
    COUNT_IF(PRODUCT_CATEGORY IS NULL) AS NULL_CATEGORIES,
```

```
    COUNT_IF(ORDER_AMOUNT IS NULL) AS NULL_ORDER_AMOUNTS,
    COUNT_IF(PAYMENT_METHOD IS NULL) AS NULL_PAYMENT_METHODS,
    COUNT_IF(ORDER_STATUS IS NULL) AS NULL_ORDER_STATUSES,
    COUNT_IF(CUSTOMER_CITY IS NULL) AS NULL_CUSTOMER_CITIES
FROM SASI_RA_DB.RAW_INGESTION.ORDERS_RAW;
```

The visible result shows zero NULL values for the displayed columns. The final result column is partly clipped in the screenshot, so verify `NULL_CUSTOMER_CITIES` in Snowsight if you need to record its exact value.

Why this matters: NULL checks help identify incomplete records before downstream transformations, reporting, or analytics. A production quality check should also consider empty strings, whitespace-only values, invalid categories, and business-specific required fields.



Check NULL values

26. Check for duplicate order IDs

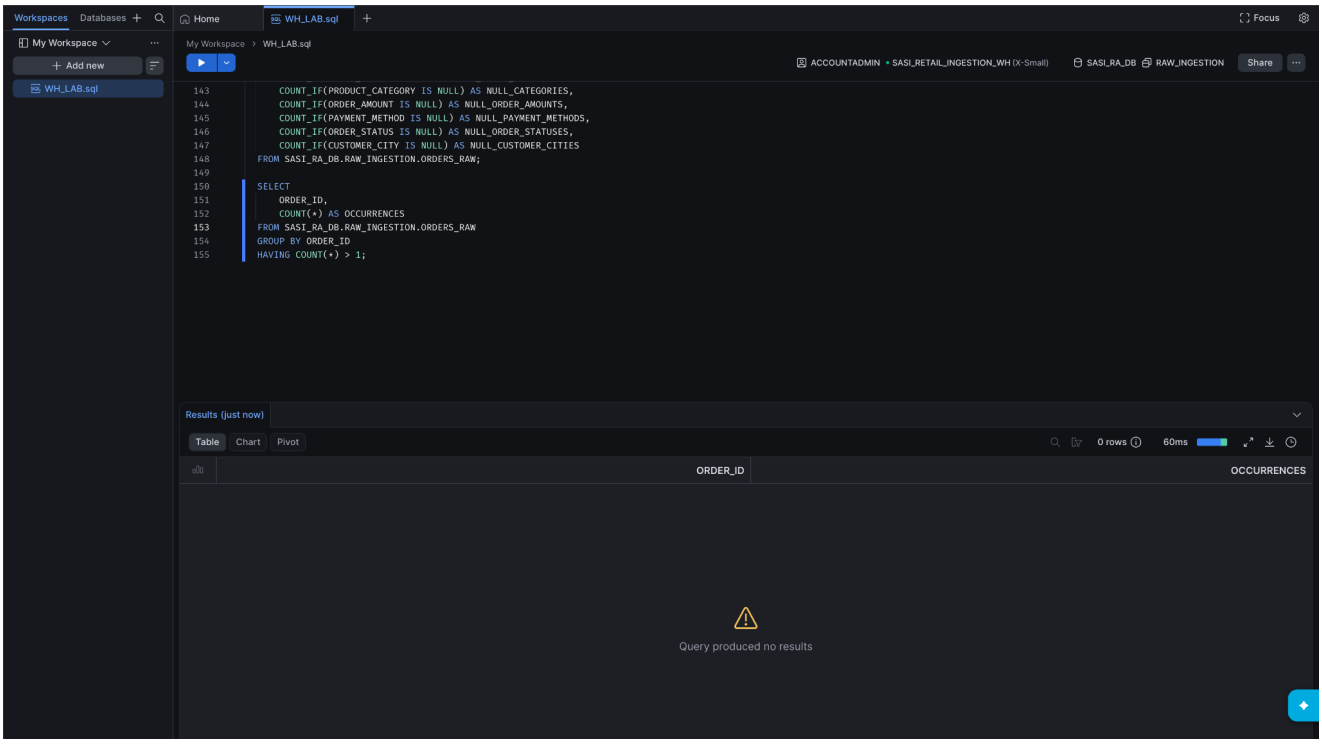
The next query groups records by `ORDER_ID` and returns only IDs that occur more than once:

```
SELECT
    ORDER_ID,
    COUNT(*) AS OCCURRENCES
FROM SASI_RA_DB.RAW_INGESTION.ORDERS_RAW
GROUP BY ORDER_ID
HAVING COUNT(*) > 1;
```

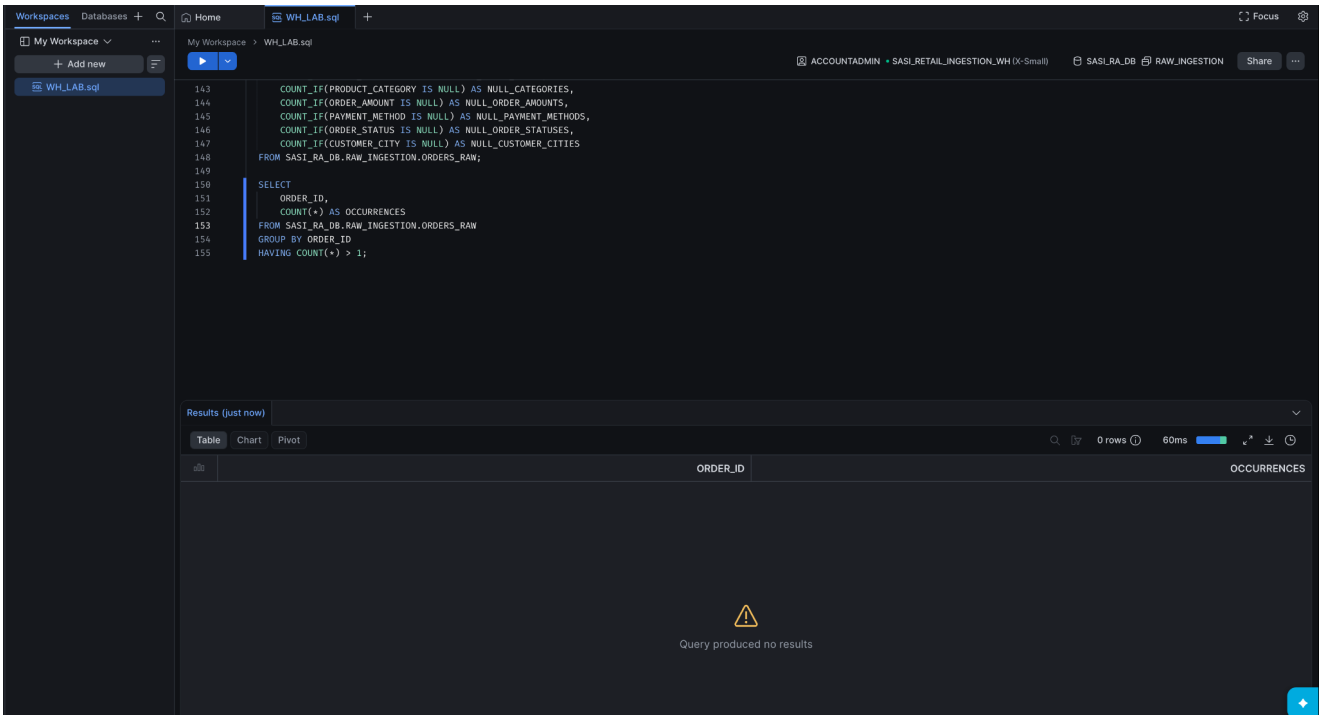
Result

The query returns **0 rows**.

That means no repeated `ORDER_ID` values were found in this 20-row dataset.



Check duplicate order IDs



Duplicate check result

Important: This is a data-quality check, not a uniqueness constraint. It verifies the current contents of the table; it does not prevent future duplicate rows from being inserted.

27. Summarize orders by order status

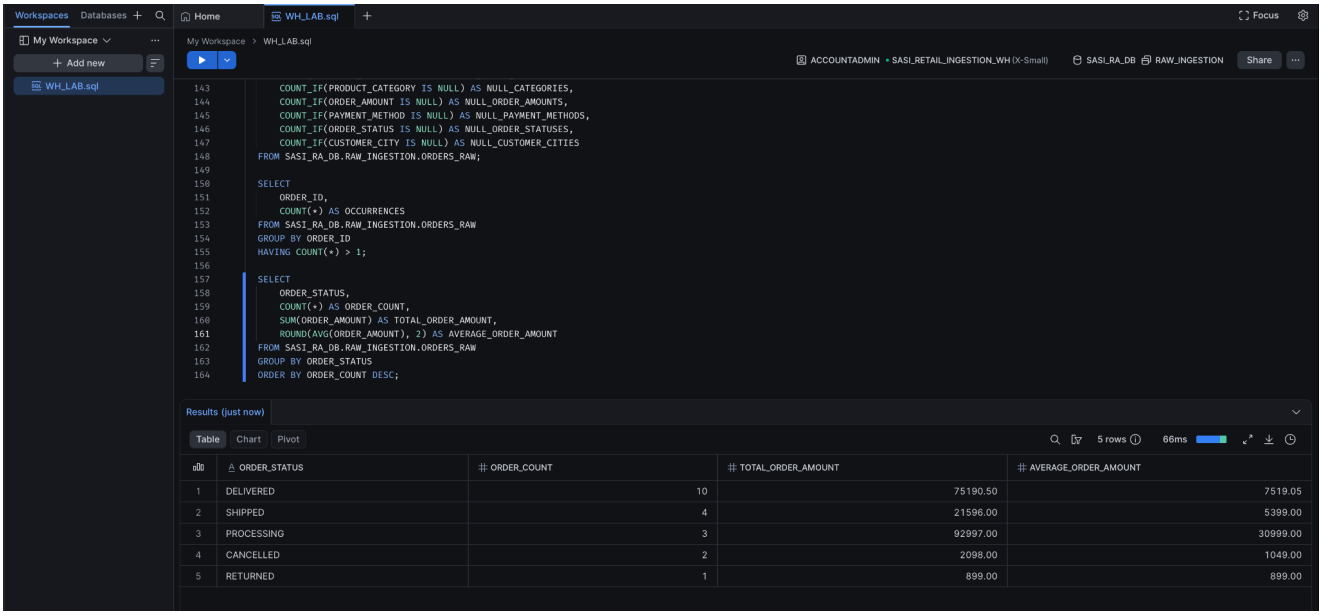
This query groups orders by their current status and calculates the number of orders, total order amount, and average order amount for each status:

```
SELECT
    ORDER_STATUS,
    COUNT(*) AS ORDER_COUNT,
    SUM(ORDER_AMOUNT) AS TOTAL_ORDER_AMOUNT,
    ROUND(AVG(ORDER_AMOUNT), 2) AS AVERAGE_ORDER_AMOUNT
FROM SASI_RA_DB.RAW_INGESTION.ORDERS_RAW
GROUP BY ORDER_STATUS
ORDER BY ORDER_COUNT DESC;
```

Observed results

Order status	Order count	Total order amount	Average order amount
DELIVERED	10	75,190.50	7,519.05
SHIPPED	4	21,596.00	5,399.00
PROCESSING	3	92,997.00	30,999.00
CANCELLED	2	2,098.00	1,049.00
RETURNED	1	899.00	899.00

There are five distinct statuses in the result. `DELIVERED` has the largest order count, while `PROCESSING` has the largest total order amount in this sample.



Order summary by status

28. Summarize orders by product category

This query calculates the number of orders and total order amount for each product category:

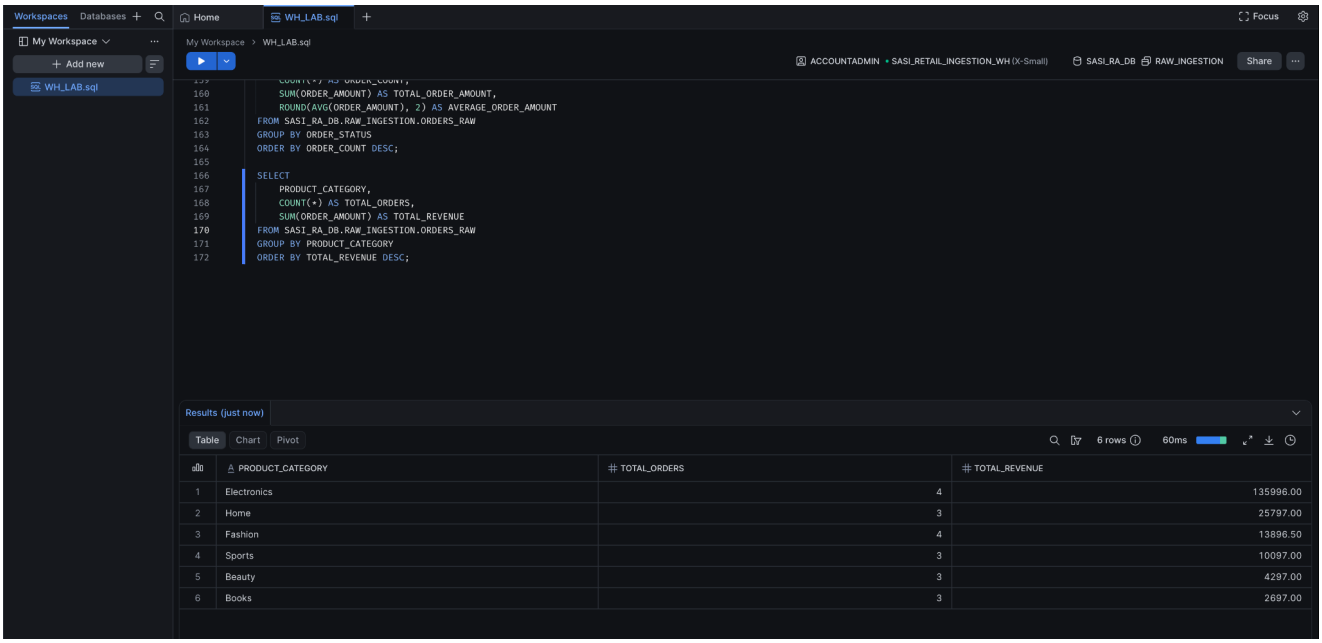
```
SELECT
    PRODUCT_CATEGORY,
    COUNT(*) AS TOTAL_ORDERS,
    SUM(ORDER_AMOUNT) AS TOTAL_REVENUE
FROM SASI_RA_DB.RAW_INGESTION.ORDERS_RAW
GROUP BY PRODUCT_CATEGORY
ORDER BY TOTAL_REVENUE DESC;
```

Observed results

Product category	Total orders	Total revenue
Electronics	4	135,996.00

Product category	Total orders	Total revenue
Home	3	25,797.00
Fashion	4	13,896.50
Sports	3	10,097.00
Beauty	3	4,297.00
Books	3	2,697.00

`Electronics` has the highest total order amount in this dataset. The query uses `SUM(ORDER_AMOUNT)`; it does not calculate profit or account for returns, discounts, taxes, or other adjustments.



Revenue summary by category

29. Inspect Snowflake COPY history

Snowflake provides load-history metadata that can be used to review recent file-loading operations.

The worksheet queries the `COPY_HISTORY` table function for the target table over the previous day:

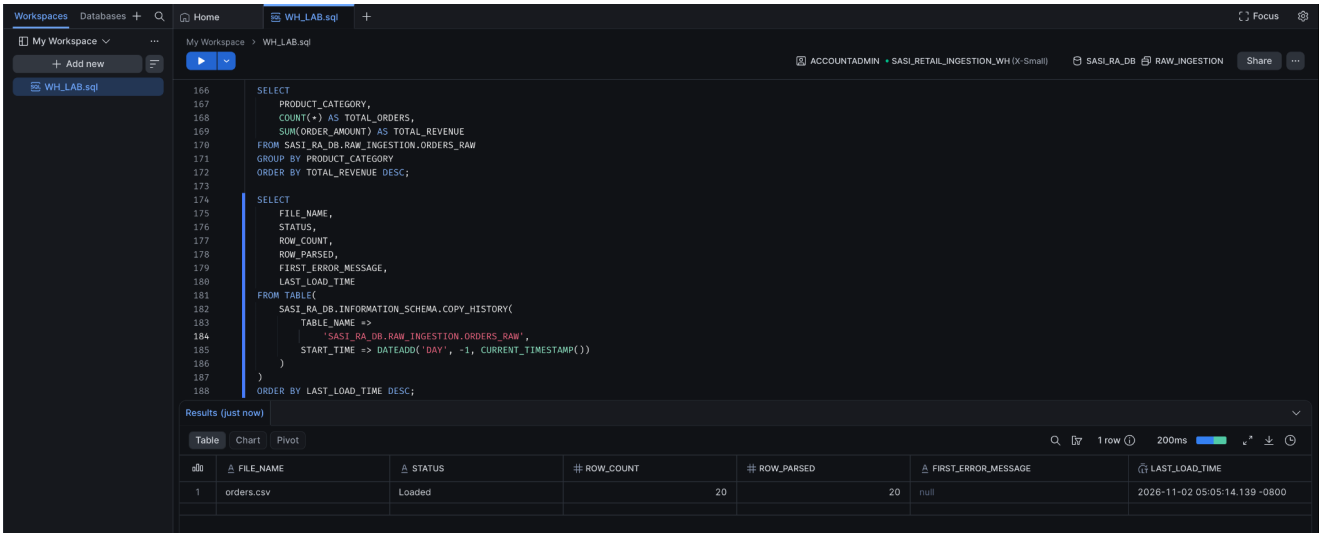
```
SELECT
    FILE_NAME,
    STATUS,
    ROW_COUNT,
    ROW_PARSED,
    FIRST_ERROR_MESSAGE,
    LAST_LOAD_TIME
FROM TABLE(
    SASI_RA_DB.INFORMATION_SCHEMA.COPY_HISTORY(
        TABLE_NAME =>
            'SASI_RA_DB.RAW_INGESTION.ORDERS_RAW',
        START_TIME => DATEADD('DAY', -1, CURRENT_TIMESTAMP())
    )
)
ORDER BY LAST_LOAD_TIME DESC;
```

Observed result

Field	Value
File name	`orders.csv`

Field	Value
Status	Loaded
Row count	20
Rows parsed	20
First error message	NULL

This gives an additional record of the ingestion operation, beyond the immediate `COPY INTO` output.



Inspect COPY history

30. Remove the staged source file

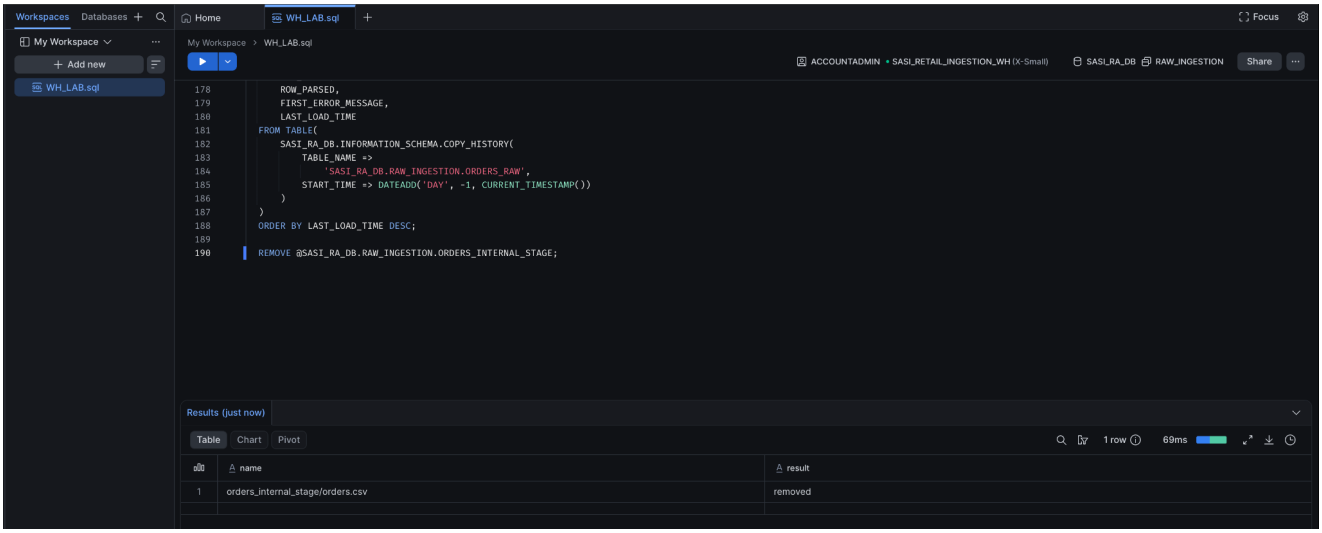
After verifying the load, the worksheet removes the uploaded file from the internal stage:

```
REMOVE @SASI_RA_DB.RAW_INGESTION.ORDERS_INTERNAL_STAGE;
```

The result reports:

```
orders_internal_stage/orders.csv    removed
```

This removes the staged file. It does **not** remove the rows already loaded into `ORDERS_RAW`.



Remove staged CSV

31. Drop the lab database

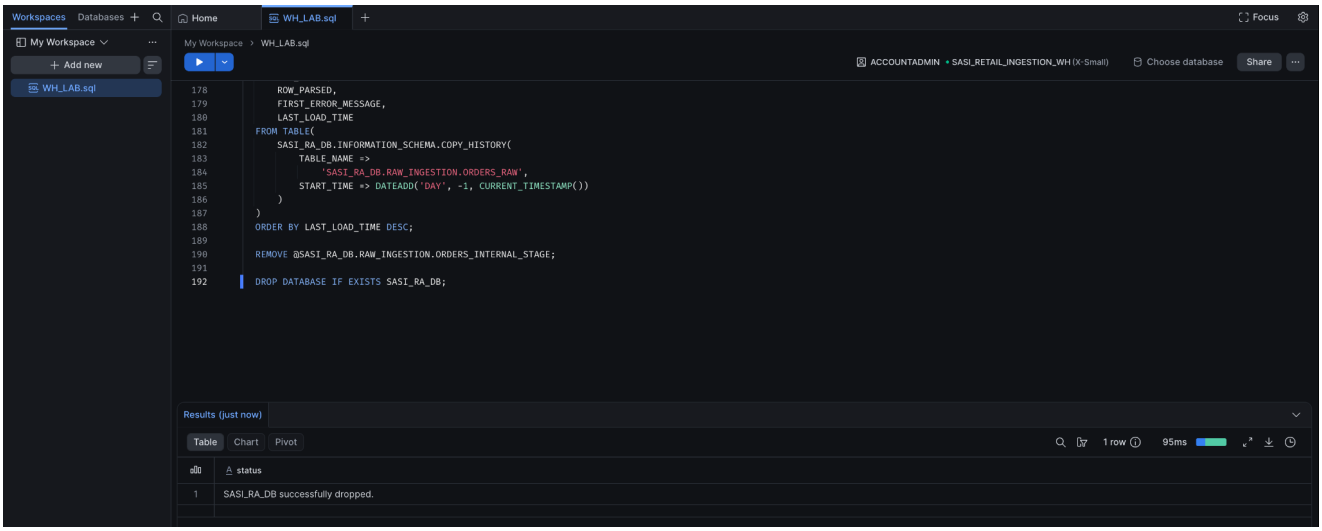
After removing the staged CSV file, the lab database was dropped:

```
DROP DATABASE IF EXISTS SASI_RA_DB;
```

Snowsight confirmed:

```
SASI_RA_DB successfully dropped.
```

This removes the database and all objects contained within it, including the `RAW_INGESTION` schema, raw table, internal stage, and file format.



SASI_RA_DB dropped successfully

32. Drop the ingestion warehouse

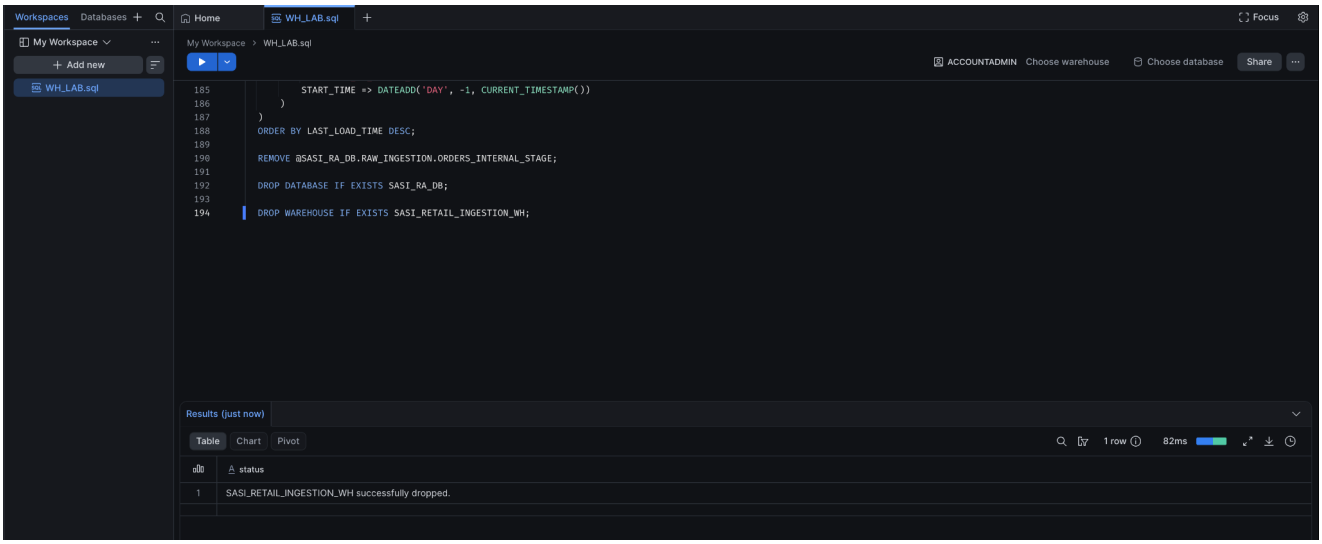
The dedicated virtual warehouse was then removed:

```
DROP WAREHOUSE IF EXISTS SASI_RETAIL_INGESTION_WH;
```

Snowsight confirmed:

```
SASI_RETAIL_INGESTION_WH successfully dropped.
```

Dropping the warehouse removes that compute resource. It does not affect other warehouses in the Snowflake account.



Ingestion warehouse dropped successfully

33. Verify that the resources were removed

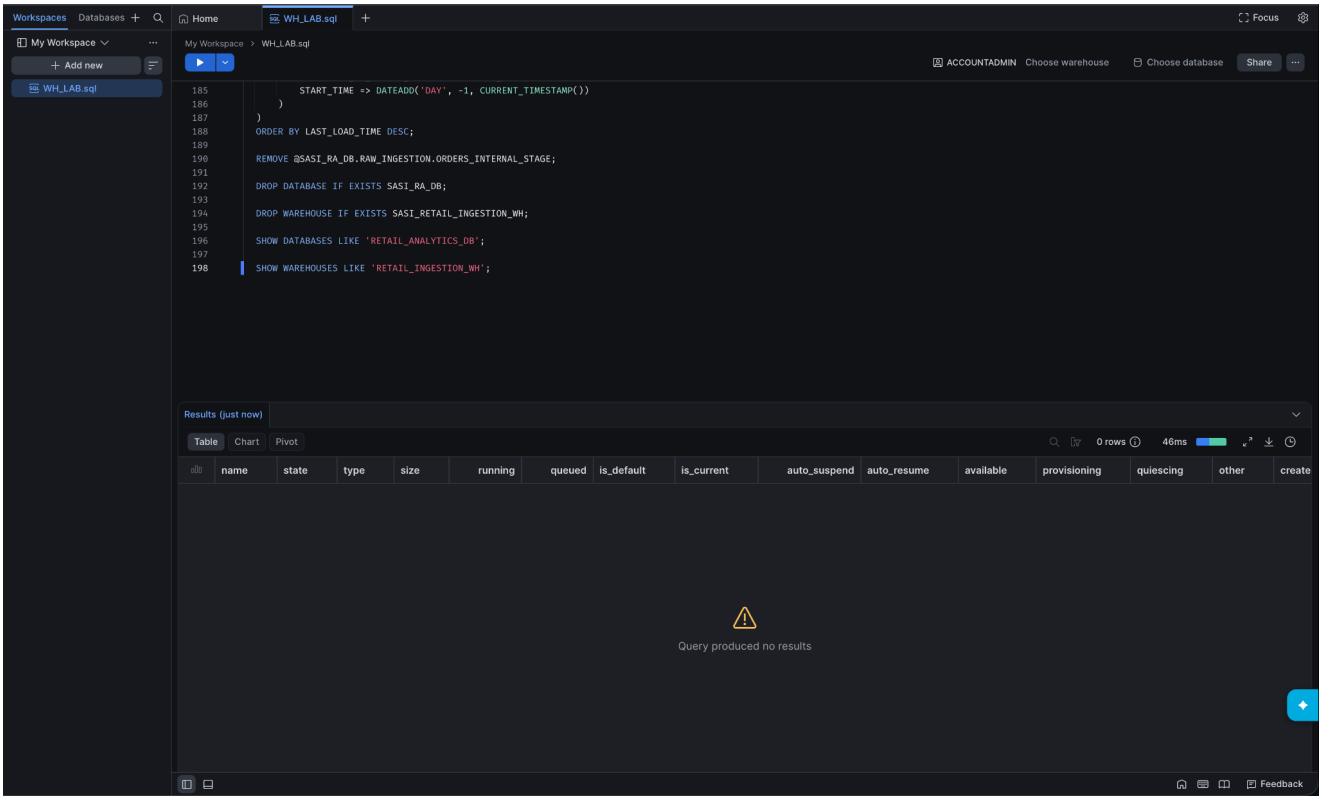
The final verification statements were:

```
SHOW DATABASES LIKE 'SASI_RA_DB';

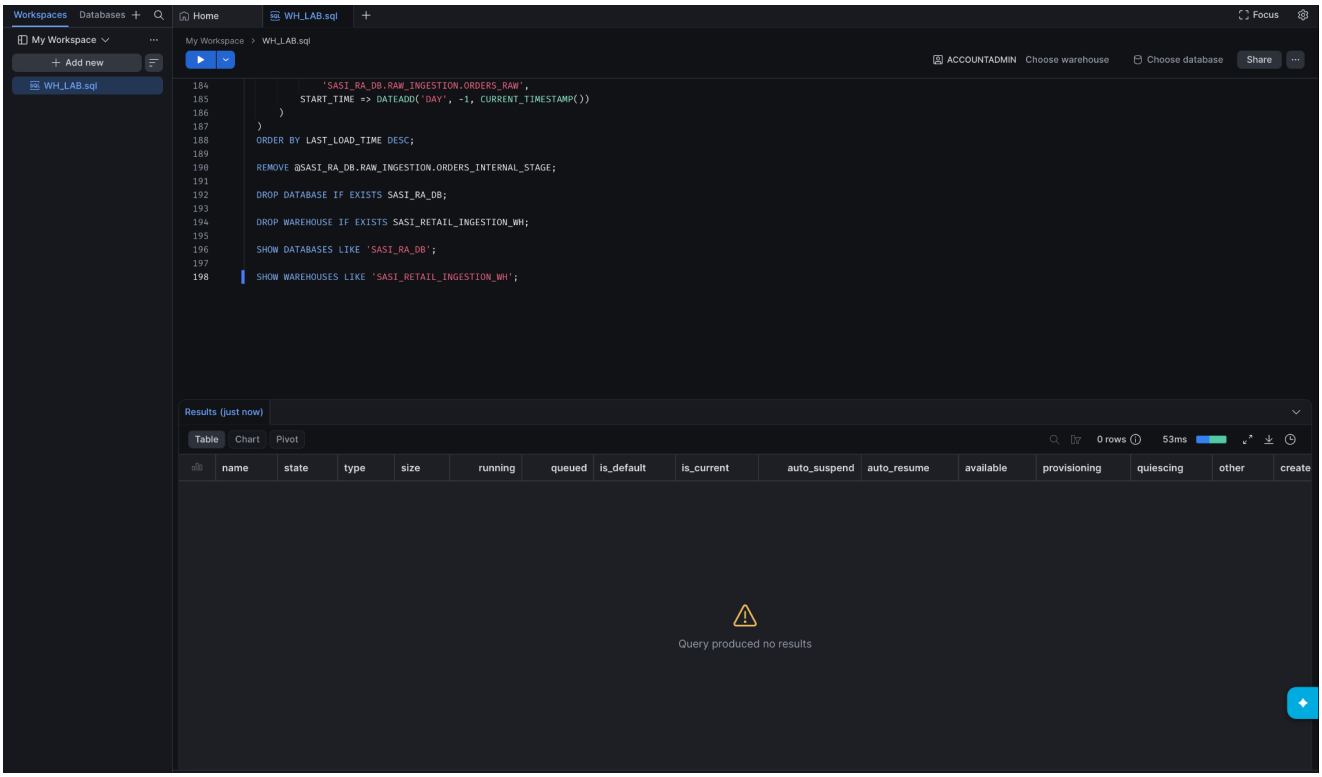
SHOW WAREHOUSES LIKE 'SASI_RETAIL_INGESTION_WH';
```

Both queries returned **0 rows** in Snowsight.

This confirms that the database and warehouse are no longer present under those names.



Verify database removal



Verify warehouse removal

34. Part 4 outcome

Check or operation	Observed result
NULL checks	Visible columns show 0 NULL values; final column is clipped
Duplicate `ORDER_ID` check	0 rows returned
Status aggregation	5 status groups
Category aggregation	6 category groups
COPY history	`orders.csv`, 20 rows, status `Loaded`
Stage file removal	`orders.csv` removed
Database cleanup	`SASI_RA_DB` successfully dropped
Warehouse cleanup	`SASI_RETAIL_INGESTION_WH` successfully dropped
Final database verification	0 rows returned
Final warehouse verification	0 rows returned

The earlier screenshot also shows a statement targeting `RETAIL_ANALYTICS_DB`, which was already dropped. The successful cleanup documented above is the later operation targeting the actual lab database, `SASI_RA_DB`.

End-to-end workflow recap

```
Upload CSV to internal stage
          |
          v
Preview and count staged records
          |
          v
COPY INTO ORDERS_RAW
          |
          v
Validate row count and sample records
          |
          v
```

```
Check NULLs and duplicate order IDs
          |
          v
Summarize orders by status and category
          |
          v
Inspect COPY_HISTORY
          |
          v
Remove staged source file
          |
          v
Verify intended database cleanup target
```

What this part demonstrates

1. Checking completeness with `COUNT_IF` and NULL predicates.
2. Checking duplicate identifiers using `GROUP BY` and `HAVING`.
3. Aggregating business measures with `COUNT`, `SUM`, and `AVG`.
4. Reviewing Snowflake ingestion metadata with `COPY_HISTORY`.
5. Removing staged files with `REMOVE`.
6. Checking database names carefully before destructive cleanup.

This documentation records the visible SQL and results from the supplied screenshots. Where a screenshot clips a result column, the limitation is explicitly noted rather than assuming its value.